



Proyecto DEMETER

Plataforma Integral de Monitorización y Gestión
para Investigación Agrícola

IoT · Firmware ESP32 · Backend FastAPI · LIMS

Documentación Técnica del Sistema

Autor: Daniel Montero Fernández

Contacto: danmonfer@us.es

Institución: Universidad de Sevilla

Versión: 1.0

Fecha: Febrero de 2026

Estado: En desarrollo activo

Este documento es confidencial y de uso interno del proyecto.

Resumen Ejecutivo

Proyecto: Demeter — Plataforma IoT para Investigación Agrícola

Autor: Daniel Montero Fernández (danmonfer@us.es)

Versión documentada: 1.0

Estado: Sistema operativo en banco; integración hidráulica y validación de campo en curso

Documento: Documentación técnica completa de arquitectura, protocolo, firmware, servicios y SDK

Propósito

Demeter es una plataforma IoT diseñada para automatizar la recolección de variables ambientales y el control de actuadores en entornos de investigación agrícola (invernaderos, cámaras de cultivo). Resuelve el problema de la *toma manual de datos* y la *intervención humana en el riego*, dos limitaciones que condicionan tanto la calidad estadística de los experimentos como la disponibilidad del personal investigador. El sistema combina tres elementos en un solo producto: **red de sensores y actuadores** sobre microcontroladores ESP32, **servicio de borde** ejecutado en una Raspberry Pi, **servidor central** con API REST y WebSocket para visualización y control en tiempo real, y un **SDK científico de Python** pensado para que investigadores puedan obtener informes de datos sin asistencia del equipo de soporte.

Arquitectura del Sistema

El sistema se descompone en cuatro capas con responsabilidades estrictamente separadas. La frontera entre capas se materializa mediante un **protocolo binario propietario** (Demeter, descrito en el Capítulo 1) entre los nodos ESP32 y la Raspberry Pi, y mediante una **API REST + WebSocket** entre la Raspberry Pi y el servidor central.

Cuadro 1: Capas del sistema Demeter y tecnologías asociadas

Capa	Función principal	Tecnología
Campo (firmware)	Sensado, actuación y enrutamiento entre nodos.	ESP32 + ESP-NOW (C++)
Borde (Raspberry Pi)	Pasarela edge: UART↔WebSocket, despacho de comandos.	Python 3 + asyncio
Servidor (backend)	API REST/WS, autenticación, persistencia, cola de cálculos pesados.	FastAPI + PostgreSQL + Redis
Cliente (frontend / SDK)	Dashboard interactivo y SDK de análisis científico para investigadores.	React (TS) + Python (<code>demeter_sdk</code>)

Decisiones arquitectónicas clave. El proyecto adopta tres compromisos de diseño que recorren toda la documentación: (i) un protocolo binario propietario en lugar de JSON sobre el enlace UART, motivado por el ancho de banda y la latencia en redes ESP-NOW; (ii) **Pydantic** como puente único entre la representación binaria del protocolo y los modelos de aplicación, evitando duplicar contratos; (iii) **Docker dual** (despliegue en GCP y Raspberry Pi local) para que el mismo *stack* se ejecute en producción y en entorno doméstico sin divergencia. Las decisiones se documentan en detalle en cada capítulo y se formalizarán como ADRs antes del cierre v1.0.

Alcance de la Versión 1.0

El alcance funcional cubierto por esta versión es deliberadamente acotado para garantizar un sistema cerrado y mantenible:

- Red de hasta 5 nodos ESP32 (sensores y actuadores) sobre ESP-NOW.
- Telemetría continua con periodicidad configurable y *Deep Sleep* para nodos sensores.
- Control remoto de bombas y actuadores discretos a través del dashboard.
- Persistencia relacional (PostgreSQL) con LIMS básico: registro botánico, experimentos y trazabilidad.
- Análisis científico desde el SDK con caché Parquet local, indicadores agronómicos (VPD, GDD, ET₀) y exportación a Excel/CSV.

Fuera de alcance v1.0 (registrado para evitar *feature creep*): app móvil, notificaciones push, capa de inferencia ML/IA, multi-tenancy, integraciones con APIs meteorológicas y refactor estructural del código existente.

Estado de Validación

El proyecto sostiene un banco de **140 tests automatizados** distribuidos en **18 archivos** sobre las cuatro capas (Capítulo 6):

Cuadro 2: Validación automatizada por capa (verificada el 2026-05-07)

Capa	Archivos	Tests	Tipo predominante
Firmware (ESP32)	5	32	Unitario (Unity + mocks)
Raspberry Pi	5	37	Mixto (asyncio mocks)
Backend (FastAPI)	4	16	Integración (TestClient + SQLite)
SDK (Python)	4	52	Unitario (funciones puras)
Frontend	2	3	3 unit.
Total	20	140	117 unit. + 23 integ.

La validación de hardware (caudal, consumo, ciclos térmicos de las bombas) estaba prevista mediante **spikes** estructurados con hipótesis verificables, y el criterio final de **DONE** para la versión 1.0 iba a ser **30 días de operación continua sin intervención manual**. Conviene decirlo de entrada: **ni el spike ni el periodo de operación llegaron a ejecutarse**. La versión 1.0 se entrega verificada en software y validada en banco, pero **sin caracterizar en campo** y sin haber actuado nunca sobre una bomba real. El Capítulo 6 detalla qué se midió y qué no, y el Capítulo 8 explica por qué.

Cómo Leer Este Documento

El documento está organizado en seis capítulos principales y tres anexos de diagramas:

- **Introducción** (visión, objetivos, alcance) y **Capítulo 1** (protocolo binario): orientación general y contrato de comunicación.
- **Capítulos de arquitectura**: firmware ESP32, servicio Raspberry Pi (**Capítulo 3**), backend (**Capítulo 4**) y SDK (**Capítulo 5**).
- **Capítulo 6**: estrategia de testing, inventario verificable de tests y métricas medidas.
- **Anexos**: diagramas de flujo del protocolo, arquitectura desplegada de la Raspberry Pi y arquitectura del backend.

Este documento se mantiene sincronizado con el repositorio del proyecto. La fecha de última verificación de los datos numéricos (tests, métricas, inventario de archivos) se indica explícitamente en cada tabla.

Índice

Resumen Ejecutivo	2
Introducción y Visión del Proyecto	9
0.1 Origen del Proyecto	9
0.2 Motivación	9
0.3 Objetivos del Sistema	9
0.3.1 Objetivo General	9
0.3.2 Objetivos Específicos	9
0.4 Alcance del Documento	10
0.5 Glosario y Siglas	10
1 Protocolo de Comunicación Demeter	12
1.1 Propósito y Filosofía de Diseño	12
1.2 Modelos Pydantic: Bridging entre Binario y JSON	12
1.2.1 Diseño de Modelos Discriminados	12
1.2.2 Auto-descubrimiento y Clasificación Automática	13
1.2.3 Conversión Binaria: Solo los Valores	13
1.3 Comunicación HTTP y WebSocket en el Sistema	13
1.3.1 Flujo de un Comando desde el Navegador	13
1.3.2 Flujo de Telemetría en Tiempo Real	14
1.4 Estructura de una Trama	14
1.5 Catálogo de Mensajes	15
1.6 Codificación de Datos	15
1.6.1 Temperatura y Humedad (Punto Fijo $\times 100$)	15
1.6.2 Duraciones en Secuencias (32 bits Little Endian)	15
1.7 Ejemplos de Tramas Reales	15
1.7.1 Telemetría de Sensor (TEMP_HUM_REPORT)	15
1.7.2 Comando de Actuación (SET_GPIO)	16
1.7.3 Inicio de Handshake (SYN)	16
1.8 Algoritmos de Serialización y Deserialización	16
1.8.1 Serialización	16
1.8.2 Deserialización y Validación	16
1.8.3 Manejo de Errores	16
1.9 Handshake TCP-like	17
1.10 Flujo Extremo a Extremo	17
1.11 Consideraciones de Tiempo y Saturación	17
1.11.1 Tiempos de Transmisión	17
1.11.2 Ciclo Energético del Nodo Sensor	17
1.11.3 Limitaciones Conocidas	18

2	Arquitectura del Firmware	19
2.1	Visión General	19
2.2	Los Tres Tipos de Nodo	19
2.2.1	Nodo Sensor	19
2.2.2	Nodo Actuador	19
2.2.3	Nodo Gateway	20
2.3	Arquitectura Interna en Capas	20
2.3.1	El Orquestador del Sistema	20
2.4	El Motor de Protocolo	21
2.4.1	Responsabilidades del Motor	21
2.4.2	Pipeline de Recepción	21
2.5	Estrategias de Transporte	21
2.5.1	Transporte ESP-NOW	21
2.5.2	Transporte UART (Serial)	22
2.5.3	Transporte Mixto (Gateway)	22
3	Servicio de Borde: Raspberry Pi	23
3.1	Misión y Posición en la Arquitectura del Sistema	23
3.2	Contenedorización y Acceso a Recursos del Sistema	23
3.3	El Motor de Concurrencia: asyncio	24
3.4	El Módulo del Protocolo Demeter en la Raspberry Pi	24
3.5	Comunicación con el Backend: El Canal WebSocket	25
3.6	El Command Dispatcher: Adaptador JSON ↔ Binario	25
3.7	Device Manager: La Caché Local de Topología	26
4	Servidor Central: Backend en Google Cloud	27
4.1	Visión General del Servidor	27
4.2	Arquitectura de Contenedores Docker	27
4.3	Redis: Bus de Mensajes y Caché	28
4.3.1	Las Cuatro Categorías de Datos en Redis	28
4.3.2	Canales Pub/Sub	28
4.4	PostgreSQL: Persistencia y Esquema de Datos	28
4.5	El Worker Matemático	29
4.6	El Secuenciador	29
4.7	El Backend FastAPI	30
4.8	El Frontend React	30
4.9	Documentación de la API REST	30
4.9.1	Sistema (/api/system)	30
4.9.2	Autenticación (/api/auth)	31
4.9.3	Comandos (/api/command)	31
4.9.4	Descubrimiento (/api/discovery)	31
4.9.5	Historial de Telemetría (/api/history)	31
4.9.6	LIMS — Plantas y Experimentos (/api/lims)	32
4.9.7	Análisis Científico (/api/analysis)	32
4.9.8	SDK Científico (/api/sdk)	32
4.9.9	Exportación (/api/export)	32
4.10	El Canal WebSocket	32

5	SDK Científico de Python — demeter_sdk	34
5.1	Propósito y Filosofía	34
5.2	Arquitectura Modular	34
5.3	Instalación y Configuración	35
5.4	Obtención de Datos: Módulo Fetcher	35
5.5	Pipeline ETL: Módulo Transform	35
5.6	Cálculos Agronómicos: Módulo Science	36
5.6.1	Termodinámica del Aire Húmedo	36
5.6.2	Fenología y Tiempo	39
5.6.3	Estadística y Detección de Anomalías	40
5.7	El Helper <code>enrich()</code> : Enriquecimiento en Una Llamada	40
5.8	Visualización Científica: Módulo Viz	41
5.9	Exportación Científica: Módulo Export	42
5.9.1	CSV	42
5.9.2	Excel Multi-hoja	42
5.10	Suite Completa — Un Solo Comando	43
5.11	Ejemplo de Sesión Completa con Jupyter	43
5.12	Referencia Rápida de Parámetros	44
6	Testing y Validación	45
6.1	Estrategia de Testing por Capa	45
6.2	Tests Unitarios	46
6.3	Tests de Integración	46
6.4	Pruebas de Campo y Validación Hardware	47
6.5	Métricas Medidas	48
6.6	Resumen del Estado de Validación	50
7	Despliegue y Operación	51
7.1	Visión General del Despliegue	51
7.2	Despliegue del Firmware ESP32	53
7.2.1	Catálogo de entornos	53
7.2.2	Procedimiento de flasheo	53
7.2.3	Verificación post-flasheo	54
7.3	Despliegue de la Raspberry Pi	54
7.3.1	Provisioning del sistema	54
7.3.2	Arranque de los servicios	55
7.3.3	Variables de entorno relevantes	55
7.3.4	Healthchecks y reinicio	55
7.4	Despliegue del Backend en GCP	55
7.4.1	Composición del stack	55
7.4.2	Procedimiento de despliegue	56
7.4.3	Reverse proxy y TLS	56
7.4.4	Variante staging con Cloudflare Tunnel	56
7.4.5	Migraciones	57
7.5	Pipeline de Integración y Entrega Continua (CI/CD)	57
7.5.1	Workflow de validación: <code>ci_develop.yml</code>	57
7.5.2	Workflow de entrega: <code>deploy_main.yml</code>	57
7.6	Operaciones Post-Despliegue	58
7.6.1	Logs	58
7.6.2	Backups de PostgreSQL	58
7.6.3	Diagnóstico y troubleshooting	58
7.7	Actualización y Rollback	59

7.7.1	Actualización	59
7.7.2	Rollback	59
7.8	Resumen	59
8	Conclusiones y Cierre de la Versión 1.0	60
8.1	Estado de la Versión 1.0 Entregada	60
8.2	Cumplimiento de Objetivos	61
8.3	Decisiones Arquitectónicas en Retrospectiva	62
8.4	Lecciones Aprendidas	63
8.5	Trabajo Futuro y Extensiones Posibles	63
8.6	Cierre y Agradecimientos	64
A	Diagramas del Protocolo Binario Demeter	65
B	Arquitectura del Servicio Raspberry Pi	75
C	Arquitectura del Backend	81

Introducción y Visión del Proyecto

0.1 Origen del Proyecto

El **Proyecto Demeter** nace en el seno de la asociación universitaria **ESIBot** (Escuela Técnica Superior de Ingeniería, Universidad de Sevilla) como un ejercicio de ingeniería full-stack aplicada a un dominio físico real: la monitorización y el control automatizado de entornos de cultivo. Su autor, Daniel Montero Fernández, conduce el proyecto en solitario a lo largo de varias iteraciones, partiendo de una primera prueba de concepto sobre Arduino y evolucionando hasta la arquitectura distribuida documentada en este texto: una red de microcontroladores ESP32, un servicio de borde sobre Raspberry Pi, un servidor central desplegable indistintamente en Google Cloud Platform o en hardware doméstico, y un SDK científico de Python para análisis agronómico.

La presente versión 1.0 marca el cierre del ciclo de desarrollo activo del proyecto. Se entrega como *release* estable y *portfolio piece*, no como abandono: el sistema queda funcional, documentado y reproducible, con una cláusula explícita de no continuidad (*no further development planned*) que libera al autor del compromiso de mantenimiento indefinido y, simultáneamente, fija el alcance de lo entregado.

0.2 Motivación

La monitorización continua de variables ambientales y la trazabilidad de los sujetos experimentales son requisitos básicos en cualquier protocolo agronómico riguroso, pero su implantación en pequeños invernaderos de investigación universitaria choca con dos limitaciones recurrentes: la **necesidad de presencia humana** para tomar lecturas y operar el riego, y el **coste de las plataformas comerciales** de gestión de información de laboratorio (LIMS), normalmente diseñadas para entornos clínicos o industriales y desproporcionadas para grupos pequeños. Demeter aborda ambos frentes con hardware de bajo coste, software libre y un alcance deliberadamente acotado.

0.3 Objetivos del Sistema

0.3.1 Objetivo General

Desarrollar una plataforma IoT integral que permita monitorizar, controlar y analizar entornos de cultivo, proporcionando trazabilidad científica completa de las plantas y los experimentos asociados.

0.3.2 Objetivos Específicos

Estos cuatro objetivos articulan el resto del documento y se retoman en el Capítulo 8 para evaluar el grado de cumplimiento de la versión 1.0:

- **Automatización IoT.** Desplegar una red de sensores de bajo coste y alta fiabilidad que recolecte variables ambientales de forma continua y sin intervención humana.

- **Control Remoto.** Permitir la actuación sobre los elementos físicos del entorno (bombas de riego, electroválvulas, ventanas) desde cualquier punto con acceso a internet.
- **Trazabilidad LIMS.** Implementar un sistema de gestión de información de laboratorio que mantenga un registro estructurado de cada planta y los experimentos en los que participa.
- **Análisis Científico.** Dotar a los investigadores de herramientas (SDK, gráficas, informes exportables) para extraer y analizar datos sin depender del equipo de soporte técnico.

0.4 Alcance del Documento

Este documento es la **documentación técnica de arquitectura** del sistema Demeter. Su audiencia objetivo son personas con perfil técnico (desarrolladores, evaluadores, mantenedores futuros) que necesitan entender *cómo está construido* el sistema, *por qué se tomaron las decisiones* de diseño que se tomaron, y *cómo desplegarlo* desde un entorno limpio.

Lo que cubre este documento:

- Arquitectura de cada una de las cuatro capas del sistema y de las fronteras entre ellas (Capítulos 1 a 5).
- Estrategia de validación y banco de tests automatizados verificable (Capítulo 6).
- Procedimiento reproducible de despliegue en los cuatro entornos soportados, incluyendo CI/CD y operaciones (Capítulo 7).
- Estado de la versión 1.0 y lecciones aprendidas (Capítulo 8).

Lo que NO cubre este documento (y dónde encontrarlo):

- **Manual de usuario final:** cómo usar el dashboard, cómo crear un experimento, cómo interpretar las gráficas. La interfaz se documenta en el propio frontend mediante *tooltips*, ayudas contextuales y un README específico del módulo.
- **Informe de viabilidad y cierre del proyecto:** análisis de sostenibilidad económica, justificación del cierre como v1.0 *release* y plan de entrega académica. Documento separado en `docs/Closure/LaTeX/main.pdf`.
- **Guías de desarrollo de plugins o módulos terceros:** la versión 1.0 no contempla extensibilidad mediante plugins. Una eventual versión futura debería añadir esta dimensión a la documentación.

0.5 Glosario y Siglas

A lo largo del documento se emplean los siguientes términos y siglas, muchos de los cuales son habituales en sus dominios respectivos pero pueden requerir contextualización conjunta.

Cuadro 3: Glosario de términos y siglas empleados en el documento

Sigla / Término	Significado y contexto en Demeter
ADR	<i>Architecture Decision Record</i> . Registro corto que documenta una decisión arquitectónica relevante, sus alternativas y su justificación.
API REST	<i>Application Programming Interface</i> sobre HTTP siguiendo el estilo arquitectónico REST.
CRC	<i>Cyclic Redundancy Check</i> . Suma de verificación que protege la integridad de las tramas del protocolo binario Demeter.
ESP-NOW	Protocolo de comunicación inalámbrica entre dispositivos ESP32 sin necesidad de un punto de acceso WiFi.
ET ₀	<i>Evapotranspiración de referencia</i> . Indicador agronómico que estima la pérdida de agua del cultivo por unidad de tiempo.
GDD	<i>Growing Degree Days</i> . Acumulado térmico que se usa para predecir fases fenológicas del cultivo.
IoT	<i>Internet of Things</i> . Paradigma de dispositivos físicos conectados que recolectan y reaccionan a datos del entorno.
JWT	<i>JSON Web Token</i> . Formato compacto de token firmado usado para autenticación entre el frontend y el backend.
LIMS	<i>Laboratory Information Management System</i> . Sistema que organiza la trazabilidad de muestras y experimentos en un laboratorio.
OTA	<i>Over-The-Air</i> . Actualización remota del firmware sin intervención física sobre el dispositivo. NO implementado en v1.0.
Parquet	Formato columnar de almacenamiento de datos usado por el SDK como caché local de telemetría.
RBAC	<i>Role-Based Access Control</i> . Modelo de control de acceso basado en roles (admin, viewer) usado por el backend.
RQ	<i>Redis Queue</i> . Librería Python que implementa una cola de tareas asíncronas sobre Redis, usada por el <i>math worker</i> .
SDK	<i>Software Development Kit</i> . En Demeter, el paquete Python <code>demeter_sdk</code> que provee acceso programático a la plataforma.
TLS	<i>Transport Layer Security</i> . Cifrado del transporte HTTPS y WebSocket Seguro (WSS).
UART	<i>Universal Asynchronous Receiver-Transmitter</i> . Enlace serie entre el ESP32 <i>gateway</i> y la Raspberry Pi.
VPD	<i>Vapor Pressure Deficit</i> . Indicador agronómico que cuantifica el estrés hídrico atmosférico sobre la planta.
WSS	<i>WebSocket Secure</i> . WebSocket sobre TLS, transporte usado entre la Raspberry Pi y el backend.

Capítulo 1

Protocolo de Comunicación Demeter

1.1 Propósito y Filosofía de Diseño

El Protocolo Demeter es un protocolo de comunicación binario diseñado específicamente para la red de campo del sistema. Los protocolos estándar de texto (JSON, CSV) son demasiado pesados para microcontroladores con memoria limitada y canales de radio estrecho. Un paquete JSON con temperatura y humedad puede ocupar entre 40 y 80 bytes. La misma información en el Protocolo Demeter ocupa exactamente **10 bytes**.

Decisión de Diseño

Las decisiones de diseño que guían el protocolo son: **compacidad** (cada bit cuenta), **determinismo** (la longitud de cada trama es conocida antes de recibirla), **robustez** (verificación de integridad en toda trama) y **extensibilidad** (hasta 254 tipos de mensaje sin modificar la estructura base).

1.2 Modelos Pydantic: Bridging entre Binario y JSON

Una aportación central de la arquitectura Demeter es que el protocolo no es exclusivamente binario: las mismas definiciones de mensaje coexisten en **dos representaciones**, gestionadas mediante **modelos Pydantic** validados y tipados:

1. **Representación JSON**: usada en la comunicación entre el Frontend y el Backend por HTTP y WebSocket. Los campos viajan con sus nombres completos (e.g. "type", "target_id", "pin", "value").
2. **Representación Binaria**: usada en la comunicación de red de campo (UART entre Raspberry Pi y Gateway, y ESP-NOW entre nodos). Solo los valores numéricos viajan en la trama; los nombres de campo *no existen en el paquete*, reduciéndolo a su mínima expresión.

1.2.1 Diseño de Modelos Discriminados

Cada tipo de comando del protocolo tiene su propio modelo Pydantic con un campo **type** que actúa como **discriminador de unión**. Esto permite que al deserializar un JSON, Pydantic identifique automáticamente de qué tipo de mensaje se trata y lo valide con el esquema apropiado:

Listing 1.1: Estructura conceptual de un modelo discriminado

```
{
  "type": "set_gpio", <-- discriminador: identifica el subtipo exacto
  "target_id": 3, <-- nodo destino
```

```
"pin": 4, <-- pin a controlar
"value": 1, <-- 1=ON, 0=OFF
"duration_ms": 15000 <-- tiempo de activacion (ms)
}
```

La clave del diseño es que el campo **type** es unívoco por subtipo. Cuando el Backend recibe este JSON (desde el Frontend o la API), instancia mediante `TypeAdapter[AnyDemeterCommand]` el modelo correcto sin lógica condicional manual.

1.2.2 Auto-descubrimiento y Clasificación Automática

El uso del patrón de **unión discriminada** con Pydantic permite que cualquier capa del sistema clasifique un mensaje desconocido sin necesidad de *switch-case* explícitos: la librería inspecciona el campo **type** y devuelve la instancia del modelo correcto, ya validada y tipada. Esto tiene tres consecuencias importantes:

- **Validación zero-cost:** si un campo falta o tiene tipo incorrecto, Pydantic lanza una excepción detallada antes de que el sistema procese nada.
- **Serialización simétrica:** `model.model_dump_json()` produce el JSON que el Frontend espera; `model.to_binary()` produce la trama que el ESP32 espera. La lógica de transformación reside en el modelo, no en el controlador.
- **Reutilización del esquema:** el mismo `Common/schemas.py` es importado por el Backend (Python) y puede referenciarse para generar SDKs cliente, documentación automática (OpenAPI) o tests de integración.

1.2.3 Conversión Binaria: Solo los Valores

La diferencia fundamental entre JSON y binario es que en la trama física **los nombres de los campos no existen**. El orden y tamaño de cada byte es implícito al tipo de comando (definido por el `CMD_ID` de la cabecera). Por ejemplo, para un `SET_GPIO` el payload es exactamente 3 bytes: `[PIN, VALUE, FLAGS]`.

La serialización binaria rellena cada byte en orden, con el tamaño y signo definidos por el esquema del protocolo (enteros sin signo, punt fijo para float, Little Endian).

1.3 Comunicación HTTP y WebSocket en el Sistema

1.3.1 Flujo de un Comando desde el Navegador

Cuando un operador interactúa con el dashboard activo, los comandos siguen un flujo completamente desacoplado de la red de campo:

1. El **Frontend React** compone un objeto JSON conforme al esquema del modelo Pydantic correspondiente (e.g. `set_gpio`) y lo envía mediante una petición **HTTP POST** a `/api/command`.
2. El **Backend FastAPI** recibe el JSON, lo valida automáticamente con el `TypeAdapter` discriminado, y publica el comando en el canal Redis `demeter:commands`. Devuelve `202 Accepted` inmediatamente.
3. El **WS Dispatcher** (tarea asyncio en el Backend) consume el mensaje de Redis y lo retransmite por WebSocket a la Raspberry Pi, identificada como `raspberry_gateway`.
4. La **Raspberry Pi** recibe el JSON por WebSocket, lo convierte a trama binaria mediante el mismo modelo Pydantic en Python, y lo envía por UART al Gateway ESP32.

5. El **Gateway ESP32** reenvía la trama binaria por ESP-NOW al nodo actuador destino, que ejecuta la acción y confirma con `CMD_ACK`.

1.3.2 Flujo de Telemetría en Tiempo Real

El camino inverso (del sensor al dashboard) funciona de forma reactiva:

1. El **Nodo Sensor** emite tramas binarias por ESP-NOW en el intervalo configurado.
2. El **Gateway ESP32** las reenvía por UART a la Raspberry Pi.
3. La **Raspberry Pi** las decodifica (Protocolo Demeter en Python), construye un JSON y lo publica por WebSocket hacia el Backend.
4. El **Backend** almacena la medición en Redis (caché 60 s) y en PostgreSQL (persistencia), y hace *broadcast* por WebSocket a todos los clientes Frontend suscritos.
5. El **Frontend** recibe el push y actualiza los gráficos sin ninguna petición adicional por parte del usuario.

Decisión de Diseño

Separación de canales: Los comandos viajan por **HTTP POST** (fiables, con respuesta de estado). La telemetría viaja por **WebSocket** (baja latencia, push). Para la Raspberry Pi, el canal WebSocket es exclusivo y bidireccional. Para el Frontend, el WebSocket es de solo lectura; sus comandos siempre van por HTTP.

1.4 Estructura de una Trama

Toda comunicación sigue el mismo formato de cabecera fija seguida de un payload variable:

Cuadro 1.1: Campos de la cabecera Demeter

Campo	Tamaño	Descripción
SYNC	1 byte	Siempre 0xAA. Marca el inicio de cualquier trama válida.
FLAGS	1 byte	Reservado. Actualmente 0x00.
SRC_ID	1 byte	Identificador numérico del nodo emisor (1–253).
DST_ID	1 byte	Destino: 0x00=servidor, 0xFF=difusión.
CMD_ID	1 byte	Tipo de mensaje (ver catálogo).
LENGTH	1 byte	Número de bytes del payload (0–255).
PAYLOAD	N bytes	Datos específicos del tipo de mensaje, formato Little Endian.
CRC	1 byte	Suma módulo 256 de FLAGS hasta fin del payload.

Tamaños: mínimo 7 bytes (sin payload) · máximo teórico 262 bytes · típico 10–14 bytes.

1.5 Catálogo de Mensajes

Cuadro 1.2: Catálogo completo de comandos del Protocolo Demeter v2

CMD_ID	Nombre	Payload (bytes)	Descripción
0x01	PING	0	Keep-alive. Espera ACK.
0x02	ACK	0–1	Confirmación con contexto de sesión opcional.
0x03	NACK	1	Rechazo con código de error.
0x04	SYN	0	Inicio del handshake (paso 1/3).
0x05	SYN-ACK	0	Confirmación de disponibilidad (paso 2/3).
0x0A	ROUTE_ADD	7	Registra una nueva ruta en la tabla del Gateway.
0x0B	TEMP_HUM_REPORT	4	Datos de temperatura y humedad (punto fijo $\times 100$).
0x0C	PIN_REPORT	2	Estado de un pin tras ejecución de un comando.
0x0D	SYSTEM_REPORT	8	Modo de operación y nivel de batería en mV.
0x10	SET_GPIO	3	Orden de cambio de estado de un pin.
0x11	SET_PWM	3	Señal analógica (control de velocidad/intensidad).
0x20	GET_SENSORS	0	Solicitud de lectura inmediata de sensores.
0x30	EXEC_SEQUENCE	Variable	Lista de pasos de actuación con temporización.

1.6 Codificación de Datos

1.6.1 Temperatura y Humedad (Punto Fijo $\times 100$)

Los valores se multiplican por 100 y se codifican como enteros con signo de 16 bits en formato Little Endian:

$$T = 24,57^{\circ}\text{C} \rightarrow 2457 = 0x0999 \rightarrow \text{bytes: [99 09]}$$

$$H = 63,20\% \rightarrow 6320 = 0x18B0 \rightarrow \text{bytes: [B0 18]}$$

1.6.2 Duraciones en Secuencias (32 bits Little Endian)

Las duraciones se expresan en milisegundos como entero de 32 bits sin signo:

$$2500\text{ ms} = 0x000009C4 \rightarrow \text{bytes: [C4 09 00 00]}$$

1.7 Ejemplos de Tramas Reales

1.7.1 Telemetría de Sensor (TEMP_HUM_REPORT)

Nodo 2 envía $T=24,57^{\circ}\text{C}$ y $H=63,20\%$ al servidor:

Listing 1.2: Trama TEMP_HUM_REPORT completa

```
Pos: [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]
Hex: AA 00 02 00 0B 04 99 09 B0 18 CRC
      SYNC FLAGS SRC DST CMD LEN T_Lo T_Hi H_Lo H_Hi
            (2) (0) TH_RPT (4) ---- 24.57 C ---- -63.20%-
```

1.7.2 Comando de Actuación (SET_GPIO)

Servidor ordena al nodo 3 encender el pin 4 (bomba):

Listing 1.3: Trama SET_GPIO

```
Pos: [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]
Hex: AA 00 00 03 10 03 04 01 00 CRC
      SYNC FLAGS SRC DST CMD LEN PIN VAL FLAGS
            (0) (3) SET_IO (3) (4) HIGH (0)
```

1.7.3 Inicio de Handshake (SYN)

Nodo sensor 2 solicita sesión con el Gateway (nodo 1):

Listing 1.4: Trama SYN mínima — 7 bytes

```
Pos: [0] [1] [2] [3] [4] [5] [6]
Hex: AA 00 02 01 04 00 CRC
      SYNC FLAGS SRC DST CMD LEN
            (2) (1) SYN (0)
```

1.8 Algoritmos de Serialización y Deserialización

1.8.1 Serialización

El proceso de serialización transforma los datos de aplicación en una trama binaria completamente estructurada. Toma el comando o reporte, convierte sus campos al formato numérico requerido (punto fijo para temperatura, Little Endian para enteros), construye el payload en un buffer, prepende la cabecera y calcula el CRC.

1.8.2 Deserialización y Validación

El *pipeline* de deserialización actúa como barrera de seguridad: verifica progresivamente el byte de inicio (SYNC), la longitud declarada, el CRC y el destino. Solo si todas las comprobaciones pasan se decodifica el payload y se lanza el evento interno asociado al tipo de mensaje.

1.8.3 Manejo de Errores

Consideración Importante

El protocolo adopta la filosofía de **fallo silencioso**: una trama corrupta no genera ninguna respuesta de error. Esto evita tormentas de retroalimentación ante interferencias de radio. El NACK se reserva para errores de lógica de aplicación (pin no válido, modo IDLE activo).

1.9 Handshake TCP-like

Antes de operar, cada nodo negocia una sesión con el Gateway mediante un triple *handshake*: el emisor envía SYN, espera SYN-ACK con *timeout* de 2 s y hasta 3 reintentos, y confirma con ACK. Solo tras este acuerdo comienzan los reportes.

1.10 Flujo Extremo a Extremo

Una trama parte del Nodo Sensor en formato binario por ESP-NOW, llega al Gateway, se reenvía por UART a la Raspberry Pi, que la decodifica y publica como JSON al Backend. El Backend la persiste en PostgreSQL y distribuye en tiempo real a todos los clientes conectados por WebSocket, con reacción visual a la décima de segundo.

1.11 Consideraciones de Tiempo y Saturación

1.11.1 Tiempos de Transmisión

Cuadro 1.3: Parámetros de tiempo de transmisión del protocolo

Parámetro	Valor
Tiempo de transmisión por trama (10B)	$\approx 0,5$ ms
Latencia extremo a extremo (RF + serial)	3 – 10 ms
Intervalo de reporte del sensor (configurado)	5.000 ms
Ocupación del canal por nodo sensor	$< 0,01\%$

1.11.2 Ciclo Energético del Nodo Sensor

El nodo sensor efectúa un intenso ahorro de recursos invocando Deep Sleep del ESP32 entre reportes. Un nodo configurado con reportes cada 60 s puede operar durante más de un año con una batería de 3000 mAh.

Consideración Importante

Los nodos actuadores y el Gateway deben permanecer activos de forma continua y requieren alimentación estable. Solo el Nodo Sensor puede operar con batería durante periodos prolongados.

1.11.3 Limitaciones Conocidas

Cuadro 1.4: Limitaciones conocidas y decisiones de diseño adoptadas

Limitación	Impacto	Decisión de diseño
Sin confirmación de entrega en ESP-NOW	Una trama puede perderse sin aviso	Aceptable para telemetría; comandos críticos usan ACK de capa de aplicación.
Máx. 250 bytes por trama ESP-NOW	Secuencias limitadas a ≈ 30 pasos por trama	Suficiente para los casos actuales; secuencias largas se pueden particionar.
Sin cifrado nativo	Tramas legibles por cualquier ESP32 cercano	AES-128 en payload es mejora planificada para v3.0.
Tabla de rutas fija (20 entradas)	Sin admisión dinámica sin reiniciar	El comando ROUTE_ADD permite registrar rutas en caliente.

Capítulo 2

Arquitectura del Firmware

2.1 Visión General

El firmware de Demeter implanta una red distribuida de microcontroladores ESP32 que operan en cooperación para cubrir tres funciones complementarias: recolectar datos del entorno, actuar sobre él y servir de puente entre el mundo físico y el sistema de información central.

Cada dispositivo ejecuta exactamente uno de tres roles, definido en tiempo de compilación: **Nodo Sensor**, **Nodo Actuador** o **Nodo Gateway**.

2.2 Los Tres Tipos de Nodo

2.2.1 Nodo Sensor

El Nodo Sensor es la unidad de observación del sistema. Su único propósito es medir el entorno y comunicar esas lecturas al resto de la red.

Capacidades:

- Mide temperatura del aire, humedad relativa y humedad del suelo.
- Opera en modo **Activo** (reporte periódico cada N segundos) o **IDLE** (escucha pasiva a petición del Gateway).
- Soporta datos simulados para desarrollo sin hardware físico.
- Realiza un proceso de establecimiento de sesión (handshake) con el Gateway antes de iniciar el envío de telemetría.
- Entre reportes entra en modo **Deep Sleep**, consumiendo μA de corriente y alargando la vida de la batería a más de un año.

Ciclo de vida: El nodo se enciende, configura sus periféricos y establece contacto con el Gateway. Una vez confirmada la sesión, alterna entre reporte y Deep Sleep.

2.2.2 Nodo Actuador

El Nodo Actuador es la unidad de acción del sistema. Recibe órdenes y las traduce en señales físicas que controlan los elementos del invernadero.

Capacidades:

- Controla pines de salida digital conectados a relés (bombas, válvulas, motores).
- Modos de seguridad: **ACTIVO** (ejecuta comandos) e **IDLE** (ignora cualquier orden, cortacircuito lógico ante mantenimiento).

- Implementa **retroalimentación confirmada**: tras ejecutar un comando envía automáticamente un informe PIN_REPORT de vuelta al servidor.
- Soporta ejecución de **secuencias multipaso temporizadas** sin bloquear el bucle principal.

2.2.3 Nodo Gateway

El Nodo Gateway es la unidad de interconexión. No mide ni actúa, sino que es el intérprete entre la red de radiofrecuencia local y la Raspberry Pi.

Capacidades:

- Habla simultáneamente ESP-NOW (RF) y UART (serial), actuando como convertidor de medio físico sin modificar el contenido de los mensajes.
- Mantiene una tabla de rutas: asocia el identificador numérico de cada nodo con su dirección MAC física.
- Reenvía comandos de la Raspberry Pi (formato binario por UART) al nodo destino correcto en la red ESP-NOW, sin interpretarlos.
- Escucha todos los mensajes de la red y los reenvía transparentemente hacia la Raspberry Pi por UART.

2.3 Arquitectura Interna en Capas

El firmware está organizado en cuatro capas claramente separadas por responsabilidad:

Cuadro 2.1: Capas del firmware Demeter

Nivel	Capa	Responsabilidad
4 (Alto)	Lógica de Nodo	Identidad y comportamiento específico del rol.
3	Orquestador	Máquina de estados, handshake, secuenciador.
2	Motor de Protocolo	Serialización/deserialización y enrutamiento.
1 (Bajo)	Estrategia de Transporte	Canal físico: ESP-NOW, UART o mixto.

2.3.1 El Orquestador del Sistema

El orquestador es el componente más complejo del firmware. Funciona como director de orquesta gestionando:

- **Máquina de estados**: BOOT → IDLE → HANDSHAKE → RUNNING → ERROR. Ninguna acción se ejecuta si el estado no lo permite.
- **Establecimiento de sesión** inspirado en TCP de tres pasos (SYN → SYN-ACK → ACK) con timeout de 2 segundos y hasta 3 reintentos.
- **Gestión de secuencias**: ejecución no bloqueante de listas ordenadas de pasos con temporización individual, evaluada en cada iteración del bucle.
- **Sistema de listeners (callbacks)**: permite suscribirse a eventos sin modificar la lógica central del orquestador.

2.4 El Motor de Protocolo

El Motor de Protocolo (Capa 2) actúa como el **corazón de la comunicación** del firmware. Es completamente agnóstico al canal físico subyacente: el mismo motor gestiona tramas tanto si provienen de ESP-NOW como de UART.

2.4.1 Responsabilidades del Motor

1. **Serialización:** transforma un comando de alto nivel (estructura de datos) en una trama binaria válida con cabecera, payload codificado en Little Endian y CRC.
2. **Deserialización:** analiza una secuencia de bytes entrante, valida el marcador de inicio (SYNC 0xAA), verifica el CRC y extrae el payload según el tipo de comando (CMD_ID).
3. **Enrutamiento:** trabaja con la tabla de rutas (ID numérico ↔ MAC física) para determinar el destinatario correcto de cada trama.
4. **Clasificación:** identifica el tipo de mensaje recibido y lanza el evento interno correspondiente (callback listener).

2.4.2 Pipeline de Recepción

Cuando el motor recibe bytes (desde cualquier transport), sigue este pipeline de validación:

1. Detección del byte de sincronía (0xAA).
2. Lectura de cabecera completa (5 bytes adicionales).
3. Verificación de que el destino (DST_ID) coincide con este nodo o es broadcast.
4. Lectura del payload de longitud declarada (LENGTH bytes).
5. Cálculo y comparación del CRC. Si no coincide: **trama descartada silenciosamente**.
6. Dispatch al listener registrado para el CMD_ID recibido.

2.5 Estrategias de Transporte

La Capa 1 del firmware abstrae el canal físico mediante el patrón **Estrategia** (*Strategy Pattern*): el Motor de Protocolo (Capa 2) no sabe nada sobre el canal físico subyacente. Simplemente llama a `transport.send(bytes)` y registra `transport.on_receive(callback)`.

Existen tres implementaciones de transporte:

2.5.1 Transporte ESP-NOW

ESP-NOW es el protocolo propietario de Espressif para comunicación directa entre microcontroladores ESP32 sin necesidad de punto de acceso Wi-Fi. Es la columna vertebral de la **red de campo** de Demeter.

Características en el contexto de Demeter:

- **Alcance:** hasta 200–400 metros en campo abierto con antenas externas.
- **Latencia:** sub-milisegundo en condiciones normales.
- **MTU:** máximo 250 bytes por trama; suficiente para cualquier trama Demeter típica.

- **Sin confirmación:** ESP-NOW no garantiza entrega; la Capa de Aplicación aporta el ACK mediante el protocolo Demeter.
- **Direccionamiento:** por dirección MAC de 6 bytes, mapeada a un ID numérico en la tabla de rutas del Gateway.

Los nodos Sensor y Actuador usan ESP-NOW como canal **exclusivo**. El Nodo Gateway usa ESP-NOW como canal de *campo* y UART como canal de *troncal*.

2.5.2 Transporte UART (Serial)

UART es la conexión física cableada entre el **Nodo Gateway ESP32** y la **Raspberry Pi**, mediante los pines TX/RX a 115200 baudios.

Características en el contexto de Demeter:

- **Fiabilidad:** al ser cableado, prácticamente sin pérdida de tramas.
- **Velocidad:** 115200 bps → capacidad de ≈ 11.500 bytes por segundo, vastamente superior a la carga de telemetría esperada.
- **Transparencia:** el Gateway reenvía las tramas binarias íntegras sin interpretarlas, actuando como pasarela pura.
- **Gestión en ambos extremos:** el ESP32 escribe/lee sobre el UART hardware; la Raspberry Pi usa `pyserial-asyncio` de forma no bloqueante.

2.5.3 Transporte Mixto (Gateway)

El Nodo Gateway implementa **dos transportes simultáneos**: escucha en ESP-NOW y retransmite los bytes recibidos por UART hacia la Raspberry, y viceversa. Esta operación de *relay* es completamente transparente: el Motor de Protocolo de la Raspberry ve las tramas como si provinieran directamente de los nodos de campo.

Decisión de Diseño

La modularidad de la capa de transporte permite añadir nuevos medios físicos (LoRa, BLE, CAN Bus) sin modificar el Motor de Protocolo ni la Lógica de Nodo. Solo se implementa la interfaz `TransportStrategy` y se registra en el orquestador.

Capítulo 3

Servicio de Borde: Raspberry Pi

3.1 Misión y Posición en la Arquitectura del Sistema

La Raspberry Pi ocupa el eslabón más crítico de toda la arquitectura Demeter: actúa como **puerta de enlace entre el mundo físico y el mundo digital**. Sin ella, no existe ningún camino técnico para que la red de sensores y actuadores ESP32 —que operan bajo un protocolo binario propietario de radiofrecuencia— se comuniquen con el servidor central en la nube.

Su responsabilidad puede enunciarse de forma simple: *escuchar la red de campo, comprender sus mensajes, y retransliterar esa información al lenguaje del servidor*; y a la inversa, *recibir órdenes del servidor y traducirlas en señales tangibles para los nodos físicos*.

Esta posición intermedia, también llamada **Edge Gateway** o Capa de Borde, es especialmente relevante en arquitecturas IoT industriales porque evita que la lógica de protocolo de campo contamine el servidor, y que el servidor requiera acceso directo al hardware.

Cuadro 3.1: Tecnologías del servicio de borde

Componente	Tecnología y función
Lenguaje UART	Python 3.9+ con modelo de concurrencia asyncio pyserial-asyncio : acceso no bloqueante al puerto serie
Red	WebSocket sobre TLS (wss://) via cable Ethernet
Despliegue	Contenedor Docker con mapeo de dispositivo físico al host
Protocolo IoT	Protocolo Demeter (propietario, binario, CRC-16)

3.2 Contenedorización y Acceso a Recursos del Sistema

El servicio Python se ejecuta íntegramente dentro de un contenedor **Docker**, lo que garantiza un entorno reproducible y aislado del sistema operativo subyacente de la Raspberry Pi (Raspberry Pi OS).

El despliegue tiene tres características relevantes:

1. **Mapeo del puerto serie**: El dispositivo físico `/dev/serial0` del host es inyectado directamente dentro del contenedor mediante la directiva de mapeo de dispositivos de Docker. Esto permite que el proceso Python en el contenedor lea y escriba en el cable UART como si estuviera en la máquina física, sin pérdida de rendimiento ni abstracción adicional.
2. **Conectividad de red**: La Raspberry Pi utiliza exclusivamente **cable Ethernet** para la conexión a internet. Esta decisión es deliberada: elimina la latencia y la inestabilidad del Wi-Fi,

especialmente relevante en entornos de invernadero con mucha interferencia de RF. El contenedor hereda la red del host de forma directa.

3. **Variables de entorno:** La URL del servidor (`WS_URL`), la velocidad del puerto serie (`BAUD_RATE`), y los credenciales de la red son inyectados como variables de entorno en tiempo de ejecución, permitiendo cambiar el entorno destino (desarrollo, producción) sin recompilar la imagen.

El Diagrama de Despliegue completo, mostrando la relación entre el host, el contenedor, el servidor y los nodos ESP32, se encuentra en la Sección B.4 del Anexo.

3.3 El Motor de Concurrencia: `asyncio`

El servicio no emplea hilos del sistema operativo (*threads*) para gestionar múltiples operaciones en paralelo. En su lugar, adopta el modelo de **concurrencia cooperativa** de Python mediante `asyncio`: un único hilo de ejecución gestiona múltiples tareas que voluntariamente ceden el control cuando están esperando una operación de entrada/salida.

Esto resulta especialmente eficiente para un servicio de este tipo, donde el 99% del tiempo de espera es I/O (esperar bytes del puerto serie, o esperar mensajes de la red).

Las cuatro tareas concurrentes que el orquestador lanza al inicio son:

Cuadro 3.2: Tareas `asyncio` del servicio Raspberry Pi

#	Tarea	Responsabilidad
1	UART RX Loop	Lee bytes del puerto serie de forma continua y no bloqueante.
2	UART TX Queue	Consume de una cola FIFO interna y escribe tramas al puerto.
3	WebSocket Client	Mantiene la conexión persistente con el servidor y envía telemetría.
4	WS Message Listener	Recibe y despacha mensajes entrantes (comandos) desde el servidor.

El flujo completo del bucle de eventos y cómo se interconectan estas cuatro tareas a través de colas asíncronas puede consultarse en la Sección B.5 del Anexo.

3.4 El Módulo del Protocolo Demeter en la Raspberry Pi

La Raspberry Pi no se limita a retransmitir bytes en bruto: **comprende el Protocolo Demeter**. Para ello, el servicio incorpora la misma lógica de serialización y deserialización descrita en el Capítulo 1, pero implementada en Python.

El módulo *UART Processor* actúa como la frontera de decodificación. Cada byte recibido pasa por un *pipeline* de validación:

1. Detección del byte de sincronía de inicio (`0xAA`).
2. Lectura de la cabecera: flags, nodo origen, nodo destino, tipo de comando y longitud.
3. Verificación del *Checksum* CRC-16 sobre el conjunto completo de la trama.
4. Desglose del *payload* según el tipo de comando reconocido.
5. Disparo del evento interno correspondiente hacia las capas superiores.

Si cualquiera de estos pasos falla, la trama es descartada silenciosamente y no se propaga ningún efecto, siguiendo la misma filosofía de fallo silente definida en el protocolo.

La información extraída de tramas válidas se clasifica según su tipo:

Cuadro 3.3: Tipos de trama reconocidos por el servicio Raspberry Pi

Tipo	Nombre	Acción desencadenada
TH_RPT	Sensor Report	Publica datos de temperatura, humedad y suelo al servidor.
SYN	Handshake Step 1	Responde con SYN-ACK y registra el nodo en el Device Manager.
ACK	Confirmación	Marca la sesión como establecida para el nodo origen.
CMD_ACK	Confirm. Actuador	Actualiza el estado del pin/actuador en la caché local.
NACK	Error de Nodo	Notifica al servidor que el comando fue rechazado por el nodo.

3.5 Comunicación con el Backend: El Canal WebSocket

La Raspberry Pi se conecta al servidor central usando un **WebSocket permanente y cifrado** sobre TLS (`wss://`), identificándose con el identificador reservado `raspberry_gateway`. Este identificador es reconocido por el servidor como la fuente de verdad de la red de campo, y tiene privilegios especiales.

Esta decisión de diseño tiene implicaciones significativas:

Decisión de Diseño

Separación de canales: El canal WebSocket es exclusivo para comunicaciones máquina a máquina (Raspberry $\leftrightarrow$ Backend). Los clientes del navegador (Frontend) usan el mismo protocolo WebSocket, pero mediante un identificador diferente y únicamente en modo *recepción*. Los comandos desde el navegador se envían como peticiones HTTP normales; el servidor los convierte en mensajes WebSocket hacia la Raspberry internamente. Esto simplifica el frontend al eliminar la lógica de reconexión y autenticación de sockets en el navegador.

La conexión WebSocket es bidireccional:

- **Dirección ascendente (RPI $\rightarrow$ Servidor):** La Raspberry publica eventos JSON para cada trama de telemetría recibida, confirmaciones de actuadores y nuevos nodos.
- **Dirección descendente (Servidor $\rightarrow$ RPI):** El servidor envía comandos JSON (`set_gpio`, `exec_sequence`, `ping`) que el Command Dispatcher traduce a tramas binarias.

El flujo ascendente se ilustra en la Sección B.2 del Anexo. El flujo descendente se ilustra en la Sección B.3 del Anexo.

3.6 El Command Dispatcher: Adaptador JSON $\leftrightarrow$ Binario

El componente más delicado de la Raspberry Pi para el flujo de comandos es el **Command Dispatcher**. Su rol es exactamente opuesto al del *UART Processor*: mientras este último convierte tramas binarias en estructuras de datos de alto nivel, el Dispatcher hace el camino inverso.

Cuando el servidor envía un comando JSON como:

Listing 3.1: Ejemplo de comando JSON recibido por la Raspberry Pi

```
{
  "type": "set_gpio",
  "target_id": 3,
  "pin": 4,
  "value": 1,
  "duration_ms": 15000
}
```

El Dispatcher realiza los siguientes pasos:

1. **Validación de esquema:** Verifica que el JSON contenga todos los campos requeridos para el tipo de comando indicado (**type**).
2. **Resolución de destino:** Consulta al Device Manager para verificar que el nodo **target_id** está registrado y activo.
3. **Empaquetación del payload:** Serializa los parámetros del comando (**pin**, **value**, **duration_ms**) siguiendo el formato binario del Protocolo Demeter.
4. **Construcción de la cabecera:** Prepende el byte de sincronía, los identificadores de origen y destino, el código de comando y la longitud.
5. **Cálculo del CRC-16:** Genera el checksum de todo el paquete.
6. **Encola en la TX Queue:** Inserta los bytes finales en la cola de transmisión del módulo UART, desde donde la tarea *TX Loop* los enviará al puerto serie en la siguiente iteración del bucle de eventos.

3.7 Device Manager: La Caché Local de Topología

Para no depender de una consulta al servidor en cada operación, la Raspberry Pi mantiene **en memoria una tabla de estado local** de todos los nodos conocidos en la red.

El *Device Manager* es el módulo responsable de esta caché. Registra:

- **Identidad del nodo:** identificador numérico y dirección MAC.
- **Instante de conexión:** timestamp del último mensaje recibido (*last_seen*).
- **Nivel de batería:** último valor reportado de tensión de la batería.
- **Estado de pines:** último estado conocido (ON/OFF) de cada salida de actuación.

Esta información permite al Dispatcher verificar si un nodo está activo antes de enviar un comando, y al *UART Processor* determinar si una trama SYN proviene de un nodo previamente conocido o de uno nuevo que requiere registro. La persistencia de esta tabla en disco (archivo **devices.json**) garantiza que tras un reinicio del contenedor, el servicio no pierde la memoria de los nodos con los que ya ha establecido sesión.

Capítulo 4

Servidor Central: Backend en Google Cloud

4.1 Visión General del Servidor

El **servidor central** de Demeter es el núcleo digital del sistema: el punto de convergencia donde la telemetría de campo adquiere persistencia, donde los experimentos científicos cobran forma, y desde donde los investigadores pueden actuar sobre el entorno físico del invernadero. Está desplegado en una **máquina virtual de Google Cloud Platform** (GCP), con acceso público protegido por TLS y orientado a múltiples clientes simultáneos.

El software del servidor está organizado en un conjunto de **contenedores Docker independientes** que colaboran a través de una red interna privada, sin puertos expuestos entre ellos al exterior, siguiendo el principio de mínimo acceso. Un **Nginx** actúa como proxy inverso único, terminando TLS y dirigiendo el tráfico al servicio correcto.

La arquitectura completa de contenedores, volúmenes y comunicaciones se muestra en la Sección C.1 del Anexo.

4.2 Arquitectura de Contenedores Docker

La pila de producción del servidor se compone de **seis contenedores** que conviven en una red privada interna denominada **demeter-net**:

Cuadro 4.1: Contenedores del sistema Demeter en producción

Contenedor	Imagen base	Escala	Función
demeter-frontend	Nginx + React build	1 instancia	SPA distribuida al navegador.
demeter-api	Python (FastAPI)	1 instancia	API REST + WebSocket + Lógica.
demeter-math-worker	Python (misma base)	Escalable	Worker de cálculos pesados.
demeter-sequencer	Python (misma base)	1 instancia	Motor de secuencias temporizadas.
demeter-db	PostgreSQL 15	1 instancia	Persistencia relacional.
demeter-redis	Redis Alpine	1 instancia	Broker, caché y cola de tareas.

Volúmenes persistentes:

- **postgres_data**: datos de la base de datos PostgreSQL.
- **shared_data**: archivos intermedios generados por el worker y accesibles desde la API (`/app/shared`).
- **export_data**: archivos de exportación de datos históricos (`/app/exports`).

Variables de entorno: cada contenedor recibe su configuración mediante un fichero `.env`, evitando credenciales en el repositorio de código. La URL de Redis, PostgreSQL, los secretos JWT y las claves de API son inyectadas en tiempo de ejecución.

4.3 Redis: Bus de Mensajes y Caché

Redis desempeña el rol más transversal del sistema: no es únicamente una caché de velocidad, sino el **bus de mensajes central** que cohesiona los distintos procesos Python que corren en contenedores independientes. Opera con dos bases de datos lógicas:

Cuadro 4.2: Bases de datos Redis y su propósito

DB	URL	Datos
DB/0	redis://...6379/0	Caché de aplicación y Pub/Sub
DB/1	redis://...6379/1	Cola de tareas RQ (<i>demeter_tasks</i>)

4.3.1 Las Cuatro Categorías de Datos en Redis

- Estado de Dispositivo** (`device:{id}:state`)
Almacena el estado ON/OFF *confirmado* de cada pin de actuador, tras recibir la confirmación de ejecución del nodo (CMD_ACK). Persiste indefinidamente hasta ser sobrescrito.
- Caché de Intención** (`demeter:cache:device:{id}`, TTL 60 s)
Cuando el usuario pulsa un botón, el estado *previsto* se almacena inmediatamente con un TTL de 60 segundos. El dashboard refleja este estado optimista antes de recibir confirmación del hardware, haciendo la interfaz percibida como instantánea.
- Telemetría** (`sensor:{id}:telemetry`, TTL 60 s)
Cada nueva trama de sensor que llega desde la Raspberry Pi es almacenada en Redis con expiración de un minuto. Endpoints de tiempo real la consultan directamente para responder en microsegundos, sin tocar PostgreSQL.
- Lote de Actividad** (`demeter:activity:batch`, cola FIFO)
Cada acción del usuario (botón pulsado, secuencia ejecutada) se encola en Redis en lugar de escribirse directamente en PostgreSQL. Una tarea en background (*Batch Flusher*) los vuelca periódicamente en la tabla `ActivityLog` de forma agrupada, evitando una avalancha de escrituras en la base de datos ante interacciones rápidas.

4.3.2 Canales Pub/Sub

Redis Pub/Sub es el mecanismo que permite a procesos en diferentes contenedores comunicarse sin acoplamiento directo:

Cuadro 4.3: Canales Pub/Sub y sus productores/consumidores

Canal	Publicador	Suscriptor
<code>demeter:commands</code>	POST /api/command	WS Dispatcher (asyncio task)
<code>iot_events</code>	RedisManager	Frontend (retransmisión WS broadcast)

El flujo completo de un comando desde el navegador hasta el ESP32, pasando por Redis y la Raspberry Pi, se muestra en la Sección C.2 del Anexo.

4.4 PostgreSQL: Persistencia y Esquema de Datos

PostgreSQL almacena toda la información de largo plazo del sistema. El esquema evoluciona mediante **Alembic**, el gestor de migraciones de SQLAlchemy, garantizando que cualquier cambio en el modelo de datos sea reproducible y trazable en todos los entornos.

Las tablas principales son:

Cuadro 4.4: Tablas principales de PostgreSQL

Tabla	Datos almacenados
users	Credenciales de acceso, roles (admin/viewer) y estado activo.
plants	Registro botánico de plantas: especie, variedad, fecha de siembra, estado vital, metadatos científicos, nodo asignado.
experiments	Agrupación de plantas en experimentos de investigación.
experimento_planta_link	Tabla intermedia N:M entre plantas y experimentos.
telemetry_th	Serie temporal de mediciones de temperatura y humedad por nodo.
activity_log	Registro de acciones (comandos enviados, interacciones de usuario).
sequences	Secuencias de pasos de riego configuradas por los usuarios.

El diagrama entidad-relación completo puede consultarse en la Sección C.3 del Anexo.

4.5 El Worker Matemático

El **Math Worker** (`demeter-math-worker`) es un proceso Python independiente que escucha la cola de tareas `demeter_tasks` en Redis (DB/1) usando la librería **RQ** (*Redis Queue*).

Su función es absorber los cálculos *pesados y no urgentes* que no deben bloquear la API principal. Las tareas que ejecuta incluyen:

- **Análisis científico por experimento:** cálculo de estadísticos de las series temporales de temperatura y humedad asociadas a un experimento (media, varianza, tendencias, correlación entre nodos).
- **Generación de informes:** empaquetamiento de los análisis en archivos de resultados exportables al volumen compartido `/app/shared`.
- **Nightly Batch:** tareas programadas de limpieza, compactación de telemetría histórica y recálculo de estadísticos agregados.

El proceso de encolar una tarea sigue este patrón: el cliente llama a `POST /api/analysis/generate`, el backend encola la tarea en Redis y devuelve un `task_id`. El cliente puede consultar `GET /api/analysis/status/{task_id}` para conocer el progreso y, una vez completada, descarga el resultado de `GET /api/analysis/download/{filename}`.

4.6 El Secuenciador

El **Secuenciador** (`demeter-sequencer`) es un proceso Python independiente con un único propósito: ejecutar secuencias de pasos de riego multi-acción de forma temporizada, sin bloquear la API ni consumir recursos del worker matemático.

Una secuencia puede definir, por ejemplo: “Abrir válvula A durante 10 segundos, esperar 2 segundos, abrir bomba B durante 30 segundos, cerrar todo”.

El Secuenciador traduce cada paso temporizado en un comando individual y lo lanza al canal Redis `demeter:commands`, que el Dispatcher convierte en una trama que llega al actuador físico. Entre paso y paso, duerme el tiempo de espera definido sin bloquear ningún otro proceso del sistema.

4.7 El Backend FastAPI

La **API principal** (`demeter-api`) es el componente central y más complejo del servidor. Implementada con el framework **FastAPI** sobre **asyncio**, combina en un único proceso el servidor HTTP REST, el servidor WebSocket, y dos tareas en background permanentes:

- **Redis Command Listener**: escucha el canal `demeter:commands` y retransmite comandos a la Raspberry Pi conectada por WebSocket.
- **Activity Batch Flusher**: cada 30 segundos vuelca el lote de eventos de actividad de Redis a PostgreSQL.

Todas las rutas que acceden a la base de datos lo hacen de forma **completamente asíncrona** mediante `AsyncSession` de SQLAlchemy, garantizando que las consultas lentas no bloqueen otras peticiones en curso.

4.8 El Frontend React

El **frontend** es una *Single Page Application* (SPA) construida en React, servida como ficheros estáticos a través de Nginx. Una vez cargada en el navegador, se comunica directamente con la API del backend mediante peticiones HTTPS y WebSocket. Ofrece las siguientes funcionalidades:

Cuadro 4.5: Módulos del frontend React

Módulo	Funcionalidades
Dashboard en tiempo real	Visualización de temperatura, humedad y suelo con gráficos actualizados vía WebSocket.
Control Manual	Botones para activar/desactivar pines de actuadores individualmente.
Planificador de Secuencias	Creación, guardado y ejecución de secuencias de riego multi-paso.
LIMS	Registro y gestión de plantas, asignación a experimentos, consulta de telemetría por planta.
SDK / Exportación	Descarga de datos en formatos académicos, acceso programático para análisis externos.

4.9 Documentación de la API REST

El servidor expone una API REST bajo el prefijo `/api`, con documentación interactiva generada automáticamente por FastAPI en `/api/docs`. A continuación se detallan todos los *routers* y sus endpoints más relevantes.

4.9.1 Sistema (`/api/system`)

Método	Ruta	Descripción
GET	/api/system/health	Comprueba el estado del servidor y la conexión a Redis y PostgreSQL. Devuelve un JSON de estado.

4.9.2 Autenticación (/api/auth)

Método	Ruta	Descripción
POST	/api/auth/login	Recibe usuario y contraseña. Devuelve un JWT Bearer Token con el rol del usuario y expiración configurable.
POST	/api/auth/register	Registra un nuevo usuario (admin). Almacena la contraseña hasheada con bcrypt.
GET	/api/auth/me	Devuelve el perfil del usuario activo autenticado mediante el token.

4.9.3 Comandos (/api/command)

Método	Ruta	Descripción
POST	/api/command	Envía un comando al hardware. El campo <code>type</code> discrimina el subtipo (<code>set_gpio</code> , <code>set_pwm</code> , <code>exec_sequence</code> , <code>ping</code> , <code>get_sensors</code>). El comando es validado con Pydantic y publicado en Redis. Devuelve 202 Accepted.

4.9.4 Descubrimiento (/api/discovery)

Método	Ruta	Descripción
GET	/api/discovery/nodes	Lista todos los nodos activos conocidos por el sistema con su último estado.
GET	/api/discovery/nodes/{id}	Estado detallado y telemetría en caché de un nodo específico.

4.9.5 Historial de Telemetría (/api/history)

Método	Ruta	Descripción
GET	/api/history/node/{id}	Devuelve hasta <code>days</code> días de telemetría histórica de temperatura y humedad para el nodo indicado. Requiere autenticación JWT.

4.9.6 LIMS — Plantas y Experimentos (/api/lims)

Método	Ruta	Descripción
GET	/api/lims/plantas	Lista todas las plantas registradas.
POST	/api/lims/plantas	Registra una nueva planta con metadatos botánicos completos.
GET	/api/lims/plantas/{id}	Detalle de una planta con sus experimentos asociados.
PATCH	/api/lims/plantas/{id}	Actualiza parcialmente los datos de una planta.
GET	/api/lims/plantas/{id}/telemetry	Telemetría de 6 meses del nodo asignado a la planta, directo de Redis.
GET	/api/lims/experimentos	Lista todos los experimentos vigentes.
POST	/api/lims/experimentos/generar	Crea un experimento y asigna plantas existentes.

4.9.7 Análisis Científico (/api/analysis)

Método	Ruta	Descripción
POST	/api/analysis/generate	Encola un análisis estadístico completo de un experimento en el Math Worker (RQ). Devuelve el <code>task_id</code> .
GET	/api/analysis/status/{id}	Consulta el estado de la tarea encolada: <i>queued</i> , <i>started</i> , <i>finished</i> o <i>failed</i> .
GET	/api/analysis/download/{f}	Descarga el archivo de análisis generado (ZIP) desde el volumen compartido.

4.9.8 SDK Científico (/api/sdk)

Método	Ruta	Descripción
GET	/api/sdk/mediciones/{exp_id}	Devuelve la telemetría cruda en JSON de todas las plantas del experimento. Protegido por X-API-Key. Prioriza caché Redis sobre consulta directa a PostgreSQL.

4.9.9 Exportación (/api/export)

Método	Ruta	Descripción
GET	/api/export/{formato}	Exporta los datos históricos en formatos CSV o XLSX para análisis externo. Se genera y devuelve como descarga directa.

4.10 El Canal WebSocket

El servidor expone el endpoint `/ws/{client_id}` como punto de entrada único para todas las conexiones WebSocket. El comportamiento varía según el identificador con el que se conecta el cliente:

Cuadro 4.6: Diferencia de comportamiento por tipo de cliente WebSocket

client_id	Tipo	Dirección	Mensajes
raspberrypi_gateway	Máquina	Bidireccional	Recibe JSON de comandos, envía JSON de telemetría.
Cualquier otro	Frontend/Monitor	Solo recepción	Solo recibe push de telemetría. Los intentos de enviar comandos son rechazados con un mensaje informativo.

Esta separación es una decisión de diseño deliberada: el canal WebSocket queda reservado para el flujo de datos en tiempo real (telemetría), mientras los comandos circulan por HTTP POST. Esto simplifica el frontend (sin lógica de reconexión de socket) y hace los comandos trivialmente verificables con herramientas estándar como `curl` o la documentación Swagger de la API.

Capítulo 5

SDK Científico de Python — `demeter_sdk`

5.1 Propósito y Filosofía

El **Demeter SDK** es la capa de acceso programático al sistema para investigadores y analistas. Nacido de la necesidad de combinar los datos de telemetría del invernadero con algoritmos agronómicos reconocidos, el SDK resuelve el ciclo completo de análisis en *una sola llamada*:

- Conectarse a la API del servidor con autenticación por clave.
- Descargar la telemetría de un experimento con caché automática.
- Limpiar y normalizar los datos (ETL).
- Calcular indicadores agronómicos derivados (VPD, GDD, ET_0 , ...).
- Exportar los resultados a CSV o Excel estructurado.
- Generar figuras de publicación científica con un único comando.

Decisión de Diseño

Filosofía de uso: el investigador no necesita conocer la arquitectura interna del sistema, ni Redis, ni WebSocket, ni la API directamente. Basta con instanciar `DemeterClient`, pasar el ID del experimento y el SDK hace el resto. El objetivo es que un científico con conocimientos básicos de Python pueda obtener un informe completo en menos de diez líneas de código.

5.2 Arquitectura Modular

El SDK está organizado en seis módulos con responsabilidades estrictamente separadas. La clase `DemeterClient` actúa como orquestador público que los une:

Cuadro 5.1: Módulos del Demeter SDK

Módulo	Rol	Clase / Función principal
<code>client.py</code>	Orquestador	<code>DemeterClient</code>
<code>fetcher.py</code>	Obtención de datos	<code>fetch_experiment()</code> , <code>fetch_plant()</code>
<code>transform.py</code>	ETL (limpieza)	<code>pipeline()</code> , <code>resample()</code>
<code>science.py</code>	Cálculos agronómicos	<code>enrich()</code> , funciones individuales
<code>viz.py</code>	Visualización científica	<code>plot_timeseries()</code> , <code>plot_vpd()</code> , ...
<code>export.py</code>	Exportación de resultados	<code>to_csv()</code> , <code>to_excel()</code>

5.3 Instalación y Configuración

El SDK se instala como un paquete Python local desde la carpeta del proyecto:

Listing 5.1: Instalación del SDK en modo editable

```
pip install -e ./SDK/
```

La autenticación se realiza mediante una **clave API** (X-API-Key) generada por el sistema. Puede pasarse directamente al constructor o mediante la variable de entorno **DEMETER_API_KEY**:

Listing 5.2: Inicialización del cliente

```
from demeter_sdk import DemeterClient

# Opción 1: API key explícita
client = DemeterClient(
    api_key = "dk_live_XXXXXXXXXXXX",
    base_url = "https://demeter.tuservidor.com",
    cache_ttl = 3600, # Caché de 1 hora
)

# Opción 2: variable de entorno DEMETER_API_KEY
client = DemeterClient(base_url="https://demeter.tuservidor.com")
```

5.4 Obtención de Datos: Módulo Fetcher

El fetcher implementa una caché transparente en formato **Apache Parquet** para evitar descargar los mismos datos repetidamente durante análisis iterativos. El comportamiento es:

- Si existe un fichero Parquet en `~/.demeter_cache/` con antigüedad inferior al TTL configurado (defecto: 1 h), se devuelve sin llamar a la API.
- Si el fichero es más antiguo o no existe, se descarga desde la API y se guarda.
- `force_refresh=True` omite la caché siempre.

Listing 5.3: Descarga de datos con caché automática

```
# Datos crudos de un experimento (últimos 30 días)
raw = client.get_raw_data(experimento_id=1, dias=30)

# Datos crudos de una planta individual (sirve el endpoint Redis)
raw_planta = client.get_plant_data(plant_id=7)

# Forzar recarga desde la API
raw_fresh = client.get_raw_data(experimento_id=1, force_refresh=True)
```

5.5 Pipeline ETL: Módulo Transform

Antes de calcular cualquier indicador, los datos crudos pasan por un pipeline de limpieza en cuatro etapas:

Cuadro 5.2: Etapas del pipeline ETL

Paso	Función	Qué hace
1	<code>to_dataframe()</code>	Convierte los JSON crudos a un DataFrame tipado. Parsea timestamps ISO-8601, coerce numérico de temperatura/humedad, ordena cronológicamente.
2	<code>clean()</code>	Elimina filas duplicadas exactas y mediciones con ambos sensores a NaN (irrecuperables).
3	<code>fill_gaps()</code>	Interpolación lineal de huecos puntuales de sensor (desconexiones breves). Soporta también método 'cubic' o 'time'.
4	<code>resample()</code>	Opcional. Agrega la serie temporal a menor frecuencia (e.g. '1h', '1D') por nodo.

Listing 5.4: Uso del pipeline ETL

```
# Pipeline completo con resampleado horario
df = client.get_clean_data(
    experimento_id=1,
    dias=30,
    resample_rule="1h", # Opcional
)
# Resultado: DataFrame con columnas [timestamp, temperature, humidity, node_id]
```

5.6 Cálculos Agronómicos: Módulo Science

El módulo `science` implementa **todas las fórmulas** agronómicas de forma vectorizada sobre NumPy/Pandas, sin bucles Python. Todas las funciones admiten arrays o `pd.Series` y devuelven `np.ndarray`.

5.6.1 Termodinámica del Aire Húmedo

Presión de Vapor de Saturación (e_s)

Calculada mediante la fórmula de **Magnus-Tetens**, el estándar de la meteorología agrícola para el rango de temperaturas habitual en invernadero (-10 a 50°C):

$$e_s = 0,6108 \cdot e^{\frac{17,27 \cdot T}{T + 237,3}}$$

In: T — Temperatura de bulbo seco ($^\circ\text{C}$).

Out: e_s — Presión de vapor de saturación (kPa).

Déficit de Presión de Vapor (VPD)

El VPD es el indicador más útil para el manejo del invernadero. Mide la diferencia entre la cantidad de agua que el aire *podría* contener (e_s) y la que *realmente* contiene ($e_a = e_s \cdot HR/100$):

$$VPD = \max(0, e_s - e_a)$$

In: e_s — Presión de vapor de saturación (kPa).

In: e_a — Presión de vapor actual (kPa). Se calcula como $e_s \cdot HR/100$.

Out: VPD — Déficit de presión de vapor (kPa).

La tabla siguiente define las zonas de riesgo para cultivos de invernadero típicos:

Cuadro 5.3: Zonas de riesgo del VPD para cultivos de invernadero

VPD (kPa)	Zona	Implicación agronómica
$< 0,4$	Riesgo fúngico	Exceso de humedad; condensación en hoja probable.
$0,4 - 1,0$	Óptimo	Transpiración activa, absorción de nutrientes eficiente.
$1,0 - 1,5$	Estrés moderado	Inicio de cierre estomático; reducir temperatura.
$> 1,5$	Estrés severo	Cierre estomático; parar riego por goteo.

Punto de Rocío (T_d)

Temperatura a la que el aire se satura si se enfría isohúidamente. Cuando la temperatura de una superficie (e.g. hoja) cae por debajo de T_d , hay condensación. Se calcula invirtiendo la fórmula de Magnus:

$$T_d = \frac{237,3 \cdot \ln(e_a/0,6108)}{17,27 - \ln(e_a/0,6108)}$$

In: e_a — Presión de vapor actual (kPa).

Out: T_d — Temperatura de punto de rocío ($^{\circ}\text{C}$).

Temperatura de Bulbo Húmedo (T_w) — Stull 2011

Temperatura límite del enfriamiento por evaporación. Usada en sistemas de refrigeración evaporativa de invernadero:

$$T_w = T \cdot \arctan\left(0,151977 \cdot \sqrt{HR + 8,313659}\right) + \arctan(T + HR) - \arctan(HR - 1,676331) + 0,00391838 \cdot HR^{1,5} \cdot \arctan(0,023101 \cdot HR) - 4,686035$$

In: T — Temperatura de bulbo seco ($^{\circ}\text{C}$).

In: HR — Humedad relativa (%).

Out: T_w — Temperatura de bulbo húmedo ($^{\circ}\text{C}$).

Humedad Absoluta

Masa de vapor de agua por metro cúbico de aire, independiente de la presión:

$$AH = \frac{2165 \cdot e_a}{T + 273,15}$$

In: e_a — Presión de vapor actual (kPa).

In: T — Temperatura ($^{\circ}\text{C}$).

Out: AH — Humedad absoluta (g/m^3).

Entalpía Específica del Aire Húmedo — ASHRAE

Energía total (térmica + latente) del aire húmedo por kilogramo de aire seco. Clave para el dimensionado de sistemas de climatización y refrigeración:

$$r = \frac{0,622 \cdot e_a}{P - e_a}$$

$$h = 1,006 \cdot T + r \cdot (2501 + 1,86 \cdot T)$$

In: T — Temperatura de bulbo seco (°C).

In: e_a — Presión de vapor actual (kPa).

In: P — Presión atmosférica (kPa, defecto 101,325).

Out: h — Entalpía específica del aire húmedo (kJ/kg aire seco).

Out: r — Relación de mezcla o humedad específica (kg agua/kg aire seco).

Índice de Calor (Rothfus, NWS)

Temperatura *percibida* por el ser humano (y por el cultivo) combinando temperatura y humedad. Solo significativo para $T \geq 27^\circ\text{C}$:

$$HI = -8,785 + 1,611 T + 2,339 HR - 0,1461 T \cdot HR$$

$$- 0,01231 T^2 - 0,01642 HR^2 + 0,002212 T^2 \cdot HR$$

$$+ 0,000726 T \cdot HR^2 - 0,000004 T^2 \cdot HR^2$$

In: T — Temperatura ambiental (°C).

In: HR — Humedad relativa (%).

Out: HI — Índice de calor percibido (°C).

Riesgo de Rocío en Hoja (LSD)

Índice que evalúa si la superficie de la hoja (modelada a $T_{leaf} = T_{air} - 2^\circ\text{C}$) está por debajo de la temperatura de punto de rocío. Un valor negativo indica condensación activa y riesgo de enfermedades fúngicas:

$$LSD = e_s(T_{leaf}) - e_a(T_{air}, HR)$$

In: T_{leaf} — Temperatura de la hoja (modelada como $T_{air} - 2$, °C).

In: e_s — Presión de vapor de saturación a T_{leaf} (kPa).

In: e_a — Presión de vapor actual del aire (kPa).

Out: LSD — Riesgo de rocío en hoja (*Leaf Surface Dew*, kPa).

5.6.2 Fenología y Tiempo

Grados-Día de Crecimiento (GDD)

El GDD es la unidad de acumulación de calor vegetal; establece el ritmo de desarrollo fenológico del cultivo. Se calcula como:

$$GDD = \max\left(0, \frac{T_{max} + T_{min}}{2} - T_{base}\right)$$

In: T_{max} — Temperatura máxima diaria (°C).

In: T_{min} — Temperatura mínima diaria (°C).

In: T_{base} — Temperatura umbral de crecimiento (°C, defecto 10).

Out: GDD — Grados-Día de Crecimiento (°C · día).

Horas de Frío (*Chill Hours*)

Recuento de horas en las que la temperatura se encuentra en el rango (0, 7,2] °C. Indicador clave para predecir la ruptura de dormancia en frutales de hoja caduca.

Evapotranspiración de Referencia — Hargreaves-Samani (ET_0)

Estima la demanda hídrica de referencia usando solo datos de temperatura, sin necesidad de piranómetros ni sensores de viento:

$$ET_0 = 0,0023 \cdot R_a \cdot (T_{mean} + 17,8) \cdot \sqrt{T_{max} - T_{min}}$$

In: R_a — Radiación extraterrestre (equivalente mm/día, defecto 15).

In: T_{mean} — Temperatura media diaria (°C).

In: T_{max} — Temperatura máxima diaria (°C).

In: T_{min} — Temperatura mínima diaria (°C).

Out: ET_0 — Evapotranspiración de referencia (mm/día).

Días Tras la Siembra (DAP)

Número de días enteros transcurridos desde la fecha de siembra:

$$DAP = \lfloor t_{sample} - t_{siembra} \rfloor$$

In: t_{sample} — Fecha de la muestra (timestamp).

In: $t_{siembra}$ — Fecha de siembra (timestamp).

Out: DAP — Días tras la siembra (días).

Permite contextualizar cada medición en términos del ciclo de vida del cultivo.

5.6.3 Estadística y Detección de Anomalías

Z-Score

Normalización lineal de una serie temporal. Detecta cuántas desviaciones estándar se aleja cada muestra de la media del periodo:

$$z = \frac{x - \mu}{\sigma}$$

In: x — Valor de la muestra.

In: μ — Media aritmética de la serie.

In: σ — Desviación estándar de la serie.

Out: z — Puntuación estándar (adim.).

El SDK aplica Z-Score de forma independiente a temperatura y humedad. Se considera outlier cualquier muestra con $|z| > 3$ (umbral configurable).

Índice de Riesgo Wallin (1962) — Tizón Tardío

Modelo clásico de epidemiología vegetal para estimación del riesgo de *Phytophthora infestans* (mildiu/tizón tardío). Evalúa un periodo horario y devuelve un nivel de riesgo:

Cuadro 5.4: Lógica del índice de riesgo Wallin

Condición	Nivel	Acción recomendada
Horas con $HR \geq 90\% < 10$	0 — Sin riesgo	Sin acción.
Horas ≥ 10 y $10^{\circ}\text{C} \leq T \leq 27^{\circ}\text{C}$	1 — Riesgo moderado	Revisión preventiva.
Horas ≥ 20	2 — Riesgo alto	Tratamiento fungicida.

Media Móvil 24h

Suavizado de la serie temporal por grupos de nodo, usando una ventana de 24 muestras (equivalente a 24 h cuando los datos son horarios). Útil para eliminar el ruido de alta frecuencia y visualizar tendencias.

5.7 El Helper `enrich()`: Enriquecimiento en Una Llamada

La función `science.enrich()` aplica en un solo paso todos los cálculos sobre el DataFrame limpio y devuelve un nuevo DataFrame con las columnas derivadas:

Cuadro 5.5: Columnas añadidas por `enrich()`

Columna	Unidad	Fórmula / Método
<code>vpd_kpa</code>	kPa	Magnus-Tetens
<code>dew_point_c</code>	°C	Inversión Magnus
<code>wet_bulb_c</code>	°C	Stull 2011
<code>abs_humidity_g_m3</code>	g/m ³	Ley del gas ideal
<code>enthalpy_kj_kg</code>	kJ/kg	Definición ASHRAE
<code>heat_index_c</code>	°C	Rothfusz (NWS)
<code>lsd_kpa</code>	kPa	Modelado hoja -2°C
<code>z_score_temp</code>	adim.	$(T - \mu)/\sigma$
<code>z_score_hum</code>	adim.	$(HR - \mu)/\sigma$
<code>temp_ma_24h</code>	°C	Media móvil 24 muestras
<code>hum_ma_24h</code>	%	Media móvil 24 muestras
<code>is_outlier</code>	bool	$ z_{temp} > 3,0$
<code>dap_days</code>	días	Solo si <code>fecha_siembra</code> provisto

Listing 5.5: Pipeline completo con enriquecimiento

```
# Datos limpios + todos los indicadores agronómicos
df = client.get_enriched_data(
    experimento_id = 1,
    dias = 30,
    fecha_siembra = "2025-09-01", # Activa columna dap_days
    resample_rule = "1h", # Opcional: resampleado horario
)

# Consultar el VPD medio de cada nodo
print(df.groupby("node_id")["vpd_kpa"].mean())

# Ver anomalías de temperatura
outliers = df[df["is_outlier"]]
print(f"{len(outliers)} lecturas anómalas detectadas")
```

5.8 Visualización Científica: Módulo Viz

Todos los gráficos están implementados con Matplotlib y Seaborn con estilo consistente y devuelven el objeto `Figure` para que el investigador pueda añadir anotaciones, cambiar el título o guardar con el DPI deseado.

Cuadro 5.6: Funciones de visualización disponibles

Función	Tipo	Descripción
<code>plot_timeseries()</code>	Serie temporal	Cualquier métrica frente al tiempo, coloreada por nodo, con media móvil superpuesta.
<code>plot_vpd()</code>	VPD con zonas	VPD en el tiempo con bandas de color que marcan las 4 zonas de riesgo agronómico.
<code>plot_boxplot()</code>	Diagrama de cajas	Distribución de cualquier métrica por nodo; ideal para comparar variabilidad entre plantas.
<code>plot_heatmap()</code>	Mapa de calor 2D	Intensidad de la métrica en el tiempo (eje X) por nodo (eje Y); resampleado diario por defecto.

Listing 5.6: Generación de gráficos

```
# Serie temporal del VPD con zonas de riesgo
fig = client.plot.plot_vpd(df)
fig.savefig("vpd_zonas.png", dpi=300, bbox_inches="tight")

# Boxplot de temperatura por nodo
fig2 = client.plot.plot_boxplot(df, metric="temperature")
fig2.savefig("boxplot_temperatura.png", dpi=300)

# Heatmap de humedad relativa (dia x nodo)
fig3 = client.plot.plot_heatmap(df, metric="humidity", resample_rule="1D")
fig3.savefig("heatmap_humidity.png", dpi=300)
```

5.9 Exportación Científica: Módulo Export

5.9.1 CSV

El fichero CSV se escribe en codificación **UTF-8 con BOM** (**utf-8-sig**) para asegurar la apertura correcta en Excel sin selección manual de codificación:

Listing 5.7: Exportación a CSV

```
client.export.to_csv(df_enriched, "resultados/telemetria.csv")
```

5.9.2 Excel Multi-hoja

El libro Excel generado tiene tres hojas estructuradas para la comunicación científica:

Cuadro 5.7: Estructura del libro Excel generado

Hoja	Contenido	Utilidad
Resumen	Media, mediana, desv. típica, mín., máx., P5, P95 de todas las métricas.	Vista rápida para el investigador principal y presentaciones.
Datos_Enriquecidos	Todos los registros con todas las columnas derivadas (13+ columnas).	Dataset completo para análisis estadístico externo (R, SPSS).
Datos_Crudos	Solo temperatura y humedad originales (sin derivados).	Reproducibilidad: los datos tal y como llegaron del sensor.

Listing 5.8: Exportación a Excel estructurado

```
client.export.to_excel(
    df_enriched,
    "resultados/informe.xlsx",
    df_raw = df_raw,
    experiment_name = "Ensayo Estrés Salino 2025",
)
```

5.10 Suite Completa — Un Solo Comando

Para el flujo de análisis estándar de un experimento, el método `run_full_suite()` ejecuta automáticamente: descarga, ETL, enriquecimiento, exportación CSV + Excel y los cuatro tipos de gráfico, guardándolo todo en una estructura de carpetas organizada:

Listing 5.9: Análisis completo en una sola llamada

```
ruta = client.run_full_suite(
    experimento_id = 3,
    dias = 60,
    output_dir = "./resultados",
    fecha_siembra = "2025-10-01",
    resample_rule = "1h",
)
# Resultado:
# resultados/Experimento_3/
# |- data/
# ||- enriched_telemetry.csv
# |+- report.xlsx
# +- plots/
# |- temperature_timeseries.png
# |- vpd_zones.png
# |- temperature_boxplot.png
# +- heatmap_temperature.png
print(f"Análisis guardado en: {ruta}")
```

5.11 Ejemplo de Sesión Completa con Jupyter

El siguiente extracto reproduciría una sesión típica de investigación en un Jupyter Notebook, desde la conexión hasta la interpretación de resultados:

Listing 5.10: Sesión completa en Jupyter Notebook

```
from demeter_sdk import DemeterClient
```

```
# 1. Conectar
client = DemeterClient(api_key="dk_live_XXX",
                       base_url="https://demeter.servidor.com")
assert client.ping(), "Servidor no disponible"

# 2. Obtener datos limpios y enriquecidos
df = client.get_enriched_data(
    experimento_id=2,
    dias=30,
    fecha_siembra="2025-09-15",
    resample_rule="30min",
)

# 3. Consultas rápidas
vpd_alto = df[df["vpd_kpa"] > 1.5]
print(f"Horas con estrés severo: {len(vpd_alto)}")

wallin = client.plot.plot_vpd(df)
wallin.savefig("vpd.png", dpi=300)

# 4. Calcular GDD acumulado del experimento
from demeter_sdk.science import gdd
df_diario = df.resample("1D", on="timestamp").agg(
    T_max=("temperature", "max"),
    T_min=("temperature", "min")
).reset_index()
df_diario["gdd"] = gdd(df_diario["T_max"], df_diario["T_min"])
df_diario["gdd_acum"] = df_diario["gdd"].cumsum()
print(df_diario[["timestamp", "gdd_acum"]].tail())

# 5. Exportar
client.export.to_excel(df, "informe_exp2.xlsx", experiment_name="Exp 2")
```

5.12 Referencia Rápida de Parámetros

Cuadro 5.8: Parámetros principales del DemeterClient

Parámetro	Tipo	Descripción
api_key	str	Clave de acceso a la API (o via DEMETER_API_KEY).
base_url	str	URL base del servidor (defecto: localhost:8000).
cache_dir	str	Ruta de la caché Parquet (defecto: ~/.demeter_cache).
cache_ttl	int	Tiempo de vida en segundos de los ficheros Parquet (defecto: 3600).
timeout	int	Timeout por petición HTTP en segundos (defecto: 30).
max_retries	int	Reintentos automáticos ante errores 5xx (defecto: 3).

Capítulo 6

Testing y Validación

6.1 Estrategia de Testing por Capa

El sistema Demeter está formado por cuatro capas con tecnologías y restricciones de ejecución radicalmente distintas: firmware C++ sobre ESP32, servicio Python asíncrono en la Raspberry Pi, backend FastAPI en GCP y SDK científico empaquetado para el usuario final. Aplicar una única política de testing a todas ellas no sería realista; en su lugar, cada capa adopta el enfoque que mejor se ajusta a sus dependencias y a la madurez de los componentes que ejerce.

Cuadro 6.1: Enfoque de testing por capa del sistema

Capa	Framework	Énfasis	Justificación
Firmware (ESP32)	Unity (PlatformIO)	Unitario	Hardware caro de simular: se aíslan controladores y motor de protocolo con mocks Arduino.
Raspberry Pi	pytest, asyncio	Mixto	Lógica de despacho concurrente: se mockean transporte UART y WebSocket para validar flujos sin red ni puerto serie.
Backend (FastAPI)	pytest, TestClient	Integración	La autenticación JWT, el RBAC y la cola RQ son emergentes: se ejercitan extremo a extremo contra una base SQLite en memoria.
SDK Científico	pytest	Unitario	Funciones agronómicas puras (VPD, entalpía, GDD): se validan con vectores numéricos de referencia, sin red.

Decisión de Diseño

Pirámide de testing pragmática. El proyecto privilegia la *base* de la pirámide — tests unitarios rápidos, deterministas y locales — y un nivel intermedio de integración acotado a fixtures en memoria. No se invierte en E2E automatizados (frontend clic-a-clic, hardware físico continuo) porque su mantenimiento no se amortiza en un sistema de un solo desarrollador. La validación E2E se delega a *spikes* manuales documentados (Sección 6.4) y a la operación real del sistema durante los 30 días de estabilización.

6.2 Tests Unitarios

La capa unitaria cubre los componentes cuyo contrato es internamente verificable: un controlador GPIO al que se le piden estados, un parser binario al que se le suministran tramas, un cálculo agronómico al que se le pasan vectores. Todos estos tests se ejecutan en menos de un segundo y no requieren hardware ni red.

Cuadro 6.2: Inventario de tests unitarios (verificado el 2026-05-07)

Capa	Archivo	#Tests	Cubre
Firmware	<code>test_gpio_controller.cpp</code>	3	Lectura/escritura de pines, pull-up, modo
Firmware	<code>test_protocol_engine.cpp</code>	25	Parseo, CRC, cabecera, callbacks (MockComms)
Firmware	<code>test_system_manager.cpp</code>	4	Inicialización, transición de estados
Raspberry Pi	<code>test_device_manager.py</code>	4	Alta, baja y consulta de nodos
Raspberry Pi	<code>test_telemetry_cache.py</code>	23	Caché en memoria, expiración, agregación
Raspberry Pi	<code>test_ws_client.py</code>	3	Reconexión, serialización de frames
SDK	<code>test_science.py</code>	23	VPD, entalpía, presión de vapor, GDD
SDK	<code>test_transform.py</code>	15	Pipeline ETL, resample, normalización
SDK	<code>test_client_offline.py</code>	7	Caché Parquet sin red
SDK	<code>test_export.py</code>	7	Exportación a CSV y Excel
Total		114	

Detalles destacables por capa:

- **Firmware.** El archivo `test_protocol_engine.cpp` concentra 25 casos sobre el parseo del protocolo binario Demeter: validación de la cabecera (Sección 1), CRC, manejo de tramas truncadas, callbacks y rutas de error. Es el componente con mayor densidad de tests porque es también el más expuesto a entradas externas.
- **Raspberry Pi.** `test_telemetry_cache.py` (23 tests) ejercita la caché que evita golpear al backend en cada lectura. Cubre expiración por TTL, agregación por nodo y comportamiento bajo carga concurrente con `asyncio`.
- **SDK.** Las funciones científicas son las que más tests soportan porque su corrección numérica es auditable: cada función agronómica se contrasta contra al menos un caso de referencia conocido (e.g. tabla de presión de vapor de saturación, ejemplos del manual de agroclimatología).

6.3 Tests de Integración

La capa de integración ejercita la *colaboración* entre componentes: una petición HTTP que atraviesa autenticación, base de datos y respuesta; un comando que viaja del despachador al transporte UART; una tarea que se encola y se procesa. Ningún test de integración requiere infraestructura real: PostgreSQL se sustituye por SQLite en memoria, Redis se mockea y el transporte UART se reemplaza por un `AsyncMock`.

Cuadro 6.3: Inventario de tests de integración

Capa	Archivo	#Tests	Ejercita
Raspberry Pi	<code>test_command_dispatcher.py</code>	3	Despachador + UART + WebSocket
Raspberry Pi	<code>test_uart_processor.py</code>	4	Transporte UART + handshake (Ping/Ack)
Backend	<code>test_auth.py</code>	6	Login, refresh token, RBAC
Backend	<code>test_command.py</code>	4	RBAC + JWT + envío de comando
Backend	<code>test_analysis.py</code>	3	Encolado en RQ, procesamiento
Backend	<code>test_lims.py</code>	3	Endpoints LIMS sobre DB en memoria
Firmware	<code>test_integration.cpp</code>	0 [†]	Bucle UART por loopback (infraestructura)
Total		23	

[†] Infraestructura de loopback presente pero sin casos `TEST_CASE` activos: queda como base para los tests Hardware-In-the-Loop previstos tras el spike de bombas (Sección 6.4).

Patrón compartido. Todos los tests de integración del backend usan un fixture `conftest.py` que arranca una aplicación FastAPI completa contra una base SQLite en memoria, crea usuarios con distintos roles y emite tokens JWT para ejercitar los endpoints. Los tests de la Raspberry Pi siguen un patrón análogo con `AsyncMock` para los componentes asíncronos (Sección 3).

Consideración Importante

Limitación conocida. Ningún test de integración cubre actualmente el flujo completo *usuario* → *backend* → *Raspberry Pi* → *ESP32* → *actuador físico*. Esa validación se hizo manualmente sobre banco, comprobando que el comando llega al nodo y conmuta el pin; nunca llegó a ejercitarse contra un actuador hidráulico real. Automatizarla exigiría un *harness* con un ESP32 dedicado conectado a la suite de tests, un coste de mantenimiento que el proyecto no asumió.

6.4 Pruebas de Campo y Validación Hardware

Las pruebas de campo cubren lo que el banco de tests automatizados no puede: el comportamiento de los componentes electromecánicos reales bajo las condiciones de operación. Estas pruebas se documentan como *spikes* con objetivos, hipótesis verificables y criterios de aceptación previos a la ejecución.

SPIKE-001 — Validación de la bomba de 5 V: planificado y no ejecutado. El primer y único *spike* del ciclo se escribió para confirmar cinco hipótesis sobre una bomba mini sumergible antes de comprometer el resto del sistema de riego:

- **H1:** la bomba entrega un caudal ≥ 50 mL/min a 5 V.
- **H2:** el consumo a régimen no excede 350 mA, dentro del margen del módulo relé.
- **H3:** el ESP32 actuador puede activar/desactivar la bomba a través del relé sin glitches.
- **H4:** el tiempo de riego típico para una maceta (50 mL) es inferior a 90 s.
- **H5:** la temperatura del cuerpo de la bomba se mantiene estable tras 6 ciclos consecutivos.

El *spike* quedó estructurado en tres sesiones de tres horas — montaje, caracterización y ciclos repetidos — con una tabla de resultados que debía decidir la continuidad: continuar, pivotar a otra bomba o abortar el rediseño físico. El procedimiento y la plantilla residen en `docs/spikes/SPIKE-001-validacion-bomba-5`

Ninguna de esas tres sesiones se llegó a celebrar. El *spike* está completo como documento y vacío como experimento: sus cinco hipótesis siguen sin verificar, y la tabla de resultados sin rellenar.

Peor aún, quedó desalineado del hardware que finalmente se adquirió: está redactado para una bomba de 5 V y la instalación comprada es de 12 V autocebada, de modo que sus umbrales de caudal y consumo ya no son aplicables. Cualquier continuación tendría que reescribirlo antes de ejecutarlo.

Se conserva aquí, sin maquillarlo, porque documenta el punto exacto en el que el proyecto dejó de avanzar sobre el mundo físico, y porque un *spike* escrito y no ejecutado es una lección de proceso más útil que su ausencia.

Validación E2E continua: no realizada. El criterio de DONE previsto para la versión 1.0 era un periodo de **30 días de operación continua sin intervención manual**, durante el cual se anotarían caídas del servicio, consumo energético, tasa de pérdida de tramas y desviación temporal del secuenciador. Ese periodo nunca se inició. **El sistema acumula cero días de operación desatendida**, y en consecuencia ninguna de esas cuatro series existe.

Pruebas de campo previas (firmware). El firmware ya ha sido sometido a pruebas de Deep Sleep en banco que validan el ciclo energético documentado en el Anexo (Figura A.8): fase activa de $\sim 1,5$ s y ventana de Deep Sleep de ~ 3596 s. Los detalles de medición están en `docs/deep_sleep_test.md`.

6.5 Métricas Medidas

Esta sección recoge únicamente métricas **efectivamente medidas** sobre el sistema. Las métricas para las que aún no existe instrumentación se marcan como pendientes para evitar inducir confianza injustificada en los datos.

Cuadro 6.4: Métricas observadas y estado de medición

Métrica	Valor	Fuente	Estado
Tiempo de fase activa nodo sensor	~1,5 s	Banco con osciloscopio	Medido
Periodo Deep Sleep nodo sensor	~3596 s	Configuración firmware + medida real	Medido
Consumo de energía en Deep Sleep	~1 μ A (orden de magnitud)	Datasheet ESP32 + observación	Medido
Latencia de comando usuario $\rightarrow$ actuador	—	Sin instrumentación	Pendiente (spike)
Throughput de tramas Gateway $\rightarrow$ RPi	—	Sin instrumentación	Pendiente
Tasa de pérdida de tramas	—	Sin instrumentación	Pendiente
Cobertura de código (todas las capas)	No formalizada	<code>pytest-cov</code> declarado en SDK, sin reporte	Pendiente

Las métricas pendientes lo siguen estando. El plan de instrumentación previsto — marcas de tiempo en frontend y ESP32 para la latencia extremo a extremo, contadores de tramas recibidas y descartadas en el servicio de borde, y ejecución de `pytest -cov` sobre backend, Raspberry Pi y SDK

— no llegó a implementarse en ninguno de sus tres puntos.

La tabla anterior queda, por tanto, como el estado definitivo de la caracterización del sistema: tres métricas medidas, todas ellas del ciclo energético del nodo, y cuatro sin instrumentar. Es poco, y decirlo importa: sin línea base medida no hay forma de demostrar que el sistema cumple sus requisitos ni de detectar una regresión. Cualquier continuación de este trabajo debería empezar por aquí, antes de añadir una sola función nueva.

6.6 Resumen del Estado de Validación

A fecha de redacción de este capítulo, el banco de tests automatizado del proyecto suma **140 tests verificables** distribuidos en **20 archivos** sobre las cuatro capas del sistema y la interfaz web:

Cuadro 6.5: Resumen agregado de tests por capa

Capa	Archivos	Tests	Unitarios	Integración
Firmware	5	32	32	0 [†]
Raspberry Pi	5	37	30	7
Backend	4	16	0	16
SDK	4	52	52	0
Frontend	2	3	3	0
Total	20	140	117	23

[†] Infraestructura de loopback presente, sin casos TEST_CASE activos.

Decisión de Diseño

Honestidad metodológica. El propósito de este capítulo no es exhibir un banco de pruebas exhaustivo, sino documentar con precisión qué se ha verificado, cómo se ha verificado y qué queda fuera. La cifra de 140 tests es modesta a escala industrial; en cambio, cubre los componentes de mayor riesgo del sistema — parser binario, despachador asíncrono, autenticación — que es donde un fallo silencioso resulta más caro.

Lo que no puede decirse es que ese banco se complementara, como estaba previsto, con validación manual estructurada y operación continua: el *spike* de hardware no se ejecutó y los 30 días de operación desatendida nunca empezaron. La versión 1.0 se entrega, por tanto, **verificada en software y sin validar en campo**. Es una distinción que este documento prefiere dejar escrita antes que difuminar.

Capítulo 7

Despliegue y Operación

7.1 Visión General del Despliegue

El sistema Demeter se despliega en **cuatro entornos diferenciados**, cada uno adaptado a un propósito distinto del ciclo de vida del proyecto. Los cuatro comparten las mismas imágenes Docker (compiladas una sola vez por la pipeline de `ghcr.io`); lo que cambia entre entornos son los *overrides* de configuración, los volúmenes y la forma de exposición al exterior.

Cuadro 7.1: Entornos de despliegue del sistema Demeter

Entorno	Propósito	Exposición	Comando
Local (dev)	Desarrollo con hot-reload, base de datos voluntal.	localhost:5173 / :8000	make dev
Staging	Pre-producción accesible vía dominio real, sin abrir puertos.	Cloudflare Tunnel a patata.monsters.org	make staging
Producción (GCP)	Servicio público estable con Traefik como proxy inverso.	patata.monsters.org	make prod (manual SSH)
Raspberry Pi	Pasarela de borde junto a los	Red local + WSS al backend	make rpi-pull

Decisión de Diseño

Una imagen, varios entornos. Las imágenes Docker se construyen una sola vez en CI (push a main) y se publican en `ghcr.io`. Cada entorno hace `pull` de la misma imagen y aplica un `docker-compose override` que cambia variables de entorno, puertos y políticas de `restart`. Esto garantiza que *lo probado en staging es bit a bit lo desplegado en producción*.

La arquitectura desplegada en GCP se ilustra en la Sección C.1 del Anexo, y el equivalente en Raspberry Pi en la Sección B.4.

7.2 Despliegue del Firmware ESP32

El firmware se construye y se sube a los nodos ESP32 mediante **PlatformIO**. La configuración de los entornos vive en `Firmware/platformio.ini` y se ejecuta desde la propia Raspberry Pi (que actúa como *flashing host* con sus tres puertos USB conectados a los ESP32 vía CDC).

7.2.1 Catálogo de entornos

Cuadro 7.2: Entornos de PlatformIO relevantes para v1.0

Env	Rol	Puerto USB	Notas
gateway	Gateway (Node 1)	/dev/ttyACM0	Receptor ESP-NOW + UART al host.
sensor_cluster	Sensor cluster (Node 2)	/dev/ttyACM3	Sensor con N plantas, ESP-NOW puro.
actuador	Actuador (Node 3)	/dev/ttyACM2	Activación de relé de bomba.
sensor_deepsleep	Sensor con Deep Sleep	/dev/ttyACM2	Variante de bajo consumo (validada en banco).
native	Tests unitarios PC	—	Banco de tests Unity (Sección 6.2).
get_mac	Utilidad	/dev/ttyACM1	Imprime la MAC del chip por serie.

El `platformio.ini` declara cuatro entornos adicionales (`sensor`, `sensor_node_3`, `sensor_test`, `integration`) usados durante el desarrollo y los tests; no forman parte del despliegue v1.0.

7.2.2 Procedimiento de flasheo

El flasheo es **USB local sobre la Raspberry Pi**. La identificación física de los puertos se realiza con `pico device list` y se contrasta con las MACs documentadas en `docs/flash_instructions.md`.

Listing 7.1: Flasheo de los tres nodos desde la Raspberry Pi

```
# 1. Identificar puertos disponibles
pico device list

# 2. Flashear los tres nodos (atajos del Makefile)
make fw-flash-gateway # /dev/ttyACM0
make fw-flash-sensor # /dev/ttyACM3
make fw-flash-actuador # /dev/ttyACM2

# 3. Equivalente sin Makefile
cd Firmware
```

```
pio run -e gateway -t upload --upload-port /dev/ttyACM0
pio run -e sensor_cluster -t upload --upload-port /dev/ttyACM3
pio run -e actuador -t upload --upload-port /dev/ttyACM2
```

7.2.3 Verificación post-flasheo

Tras el flasheo, cada nodo debe emitir una traza identificable por el monitor serie:

Listing 7.2: Verificación serie del gateway

```
make fw-monitor-gateway
# Salida esperada:
# === DEMETER GATEWAY V2 (Direct Control) ===
# [Setup] Ready.
# >> [CLUSTER] Node 2: 4 entries
# Plant 1: 22.50 C, 45% soil
```

Consideración Importante

OTA no implementado en v1.0. Las actualizaciones de firmware requieren acceso físico a los puertos USB de la Raspberry Pi. Esta es una limitación deliberada: implementar *Over-The-Air* sobre ESP-NOW exige reservar particiones, gestionar autenticación y manejar *rollback* en caso de fallo, trabajo que no se justifica para un sistema doméstico de un solo despliegue. Si en una versión futura se distribuyera a múltiples invernaderos, OTA se convierte en un requisito.

7.3 Despliegue de la Raspberry Pi

La Raspberry Pi cumple dos funciones simultáneas: **pasarela de borde** (*edge gateway*) entre los ESP32 y el backend, y opcionalmente **servidor de UI local** para usuarios sin acceso a internet pero con pantalla conectada al dispositivo.

7.3.1 Provisioning del sistema

Sobre una Raspberry Pi OS recién instalada, los pasos para preparar el host son:

Listing 7.3: Provisioning de la Raspberry Pi

```
# 1. Instalar Docker y plugin compose
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER && newgrp docker

# 2. Liberar el UART GPIO: deshabilitar consola serie del kernel
sudo raspi-config nonint do_serial 1 # console=disabled, hw=enabled
# Equivalente manual:
# - Editar /boot/firmware/cmdline.txt y eliminar 'console=serial0,115200'
# - Asegurar 'enable_uart=1' en /boot/firmware/config.txt
sudo reboot

# 3. Verificar que /dev/serial0 existe y el usuario pertenece a 'dialout'
ls -l /dev/serial0
groups | grep dialout || sudo usermod -aG dialout $USER

# 4. Clonar repositorio
git clone https://github.com/<usuario>/Proyecto_Demeter.git
cd Proyecto_Demeter
cp .env.rpi.example .env.rpi # ajustar DEMETER_BACKEND_URL si fuera distinto
```

7.3.2 Arranque de los servicios

El despliegue en RPi se gestiona con `docker-compose.rpi.yml`, que define dos servicios: `gateway` (FastAPI + bridge UART↔WS) y `local-ui` (Nginx sirviendo la SPA React). Ambos corren en `network_mode: host` para evitar la sobrecarga del NAT de Docker en hardware ARM modesto.

Listing 7.4: Arranque del stack en la Raspberry Pi

```
# Imagen pre-compilada (multi-arch arm64) desde GHCR (recomendado)
make rpi-pull

# 0 construir localmente si se han hecho cambios sin pasar por CI
make rpi-build

# Operación cotidiana
make rpi-logs # logs en vivo
make rpi-status # estado de contenedores y env vars activas
make rpi-shell # entrar al contenedor 'demeter-gateway'
```

7.3.3 Variables de entorno relevantes

Cuadro 7.3: Variables de entorno consumidas por el contenedor `gateway`

Variable	Función	Default
DEMETER_BACKEND_URL	Host del backend (sin esquema).	patata.monsters.org
DEMETER_BACKEND_PORT	Puerto WSS hacia el backend.	443
DEMETER_PORT	Path del dispositivo serie dentro del contenedor.	/dev/ttyS0
DEMETER_UART_BAUD	Velocidad UART (debe coincidir con firmware Gateway).	115200
DEMETER_LOG_LEVEL	Verbosidad (DEBUG, INFO, ...).	INFO
HOST_SERIAL_PORT	Mapeo del dispositivo del host (/dev/serial0).	/dev/serial0

7.3.4 Healthchecks y reinicio

A diferencia del backend (Sección 7.4), los servicios de la Raspberry Pi **sí incluyen healthchecks** en `docker-compose.rpi.yml`: el `gateway` expone `http://localhost:8001/api/health` y el `local-ui` responde en el puerto 80. La política de restart es `unless-stopped`, suficiente para reanudar tras cortes de luz frecuentes en entornos no controlados.

7.4 Despliegue del Backend en GCP

El backend se despliega como un **stack Docker Compose** sobre una máquina virtual de Google Cloud Platform. La VM no se aprovisiona automáticamente: se asume creada manualmente con Ubuntu LTS y Docker instalado, y con el dominio `patata.monsters.org` apuntando a su IP pública.

7.4.1 Composición del stack

El stack en producción combina dos archivos Compose:

- `docker-compose.yml` (base): define los seis servicios (`frontend`, `api`, `math-worker`, `sequencer`, `db`, `redis`) descritos en el Capítulo 4.
- `docker-compose.prod.yml` (override): aplica `restart: always` a todos los servicios y añade `labels` de Traefik para enrutar `patata.monsters.org` y sus paths `/api/` y `/ws/`.

7.4.2 Procedimiento de despliegue

El target `prod` del Makefile no orquesta el despliegue automáticamente: imprime el procedimiento manual a seguir por SSH. Esta decisión es deliberada para evitar despliegues accidentales contra producción desde un portátil de desarrollo.

Listing 7.5: Procedimiento de despliegue en producción

```
# 1. Conexión al servidor
ssh danielmf31@patata.monsters.org

# 2. En el servidor: actualizar código y configuración
cd ~/Proyecto_Demeter
git pull origin main
cp .env.prod .env

# 3. Levantar stack con override de prod
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build

# 4. Migraciones (idempotente)
docker exec demeter-api sh -c "cd /app/Software/Servidor/Backend && alembic upgrade head"

# 5. Seed inicial (solo en el primer despliegue)
docker exec demeter-api python -m BD.seed_data
```

7.4.3 Reverse proxy y TLS

Traefik actúa como proxy inverso único frente al stack. Termina TLS para `patata.monsters.org` (certificado gestionado por Let's Encrypt vía Traefik ACME) y enruta:

- `patata.monsters.org/` → `demeter-frontend:80`
- `patata.monsters.org/api/...` → `demeter-api:8000`
- `patata.monsters.org/ws/...` → `demeter-api:8000` (WebSocket)

Traefik no se incluye en `docker-compose.prod.yml`: se asume desplegado a nivel de la VM, escuchando en `:80` y `:443` y conectado a la misma red Docker que el stack Demeter mediante una red externa compartida.

7.4.4 Variante staging con Cloudflare Tunnel

Para validar cambios contra el dominio público sin abrir puertos en el portátil de desarrollo, `docker-compose.staging` añade un servicio `cloudflared` que crea un túnel Cloudflare Zero Trust:

Listing 7.6: Variante staging con túnel Cloudflare

```
# .env.staging debe definir CLOUDFLARE_TUNNEL_TOKEN obtenido en
# dash.cloudflare.com -> Zero Trust -> Tunnels.
make staging

# Acceso:
# https://patata.monsters.org (vía túnel)
# http://localhost:5175 (acceso directo sin túnel)
# http://localhost:8001/api/docs (Swagger directo)
```

Este modo es útil para demostraciones desde el portátil sin tocar la VM de producción y para sesiones de QA en las que se quiere ejercitar el sistema con un certificado TLS real sin comprometer datos productivos.

7.4.5 Migraciones

Las migraciones de PostgreSQL se gestionan con **Alembic**. El target `make migrate` ejecuta `alembic upgrade head` dentro del contenedor `demeter-api`. Las migraciones **inversas (downgrade) no están soportadas oficialmente**: ante un fallo de migración se recomienda restaurar desde backup en lugar de revertir esquema.

7.5 Pipeline de Integración y Entrega Continua (CI/CD)

El proyecto cuenta con dos workflows de GitHub Actions ya operativos en `.github/workflows/`, uno para validación continua y otro para construcción y publicación de imágenes Docker.

7.5.1 Workflow de validación: `ci_develop.yml`

Disparado en `push` y `pull_request` contra la rama `develop`. Su único job, `test-all`, prepara un entorno con Python 3.12 y Node 20, instala las dependencias de las cuatro suites del proyecto y ejecuta el target `make test-all` (Capítulo 6). Aplica caché para `pip` y `~/.platformio` para acelerar las ejecuciones sucesivas.

7.5.2 Workflow de entrega: `deploy_main.yml`

Disparado en `push` a la rama `main`. Ejecuta primero `test-all` como gate y, si pasa, lanza el job `build-and-push` con una estrategia de matriz que construye **cuatro imágenes Docker multi-arquitectura**:

Cuadro 7.4: Imágenes Docker publicadas por la pipeline

Imagen	Plataforma	Dockerfile	Build args
frontend	linux/amd64	Software/Servidor/docker/frontend/Dockerfile	VITE_API_URL, VITE_WS_URL, VITE_DEV_LOGIN
backend	linux/amd64	Software/Servidor/docker/backend/Dockerfile	—
gateway	linux/arm64	Software/Raspberry/Dockerfile	—
local-ui	linux/arm64	Software/Servidor/docker/frontend/Dockerfile.rpi	VITE_API_URL=/api, VITE_WS_URL=/ws

Cada imagen se etiqueta con cinco patrones simultáneos: nombre de rama, hash corto de commit, `latest`, versión semver (si el commit lleva tag) y la etiqueta fija `0.4` (versión de la pipeline). Las imágenes ARM64 se construyen sobre runners x86 mediante `QEMU`, evitando depender de runners self-hosted en Raspberry Pi.

Consideración Importante

Lo que el CI no hace. La pipeline construye imágenes pero **no despliega automáticamente**: el pull desde `ghcr.io` y el arranque del stack se realiza manualmente desde la VM o la Raspberry Pi. La razón es operativa: en un sistema con cinco macetas reales, una promoción automática a producción durante un experimento en curso puede invalidar los datos del investigador. Esta es una decisión deliberada, no una deuda técnica.

Tampoco se construye firmware en CI ni se ejecuta análisis de secretos (`gitleaks`); ambas tareas están planificadas para versiones futuras.

7.6 Operaciones Post-Despliegue

Una vez el sistema está corriendo, las operaciones recurrentes (logs, backups, diagnóstico) se realizan vía Makefile o vía `docker compose` directamente.

7.6.1 Logs

Listing 7.7: Atajos de logs por servicio

```
make logs # Todos los servicios del stack activo (tail 50)
make logs-api # Solo API FastAPI
make logs-db # Solo PostgreSQL
make logs-redis # Solo Redis
make logs-frontend # Solo Nginx (frontend)
make rpi-logs # Stack Raspberry Pi (gateway + local-ui)
```

No hay centralización de logs (Loki, ELK, Cloud Logging). Para un sistema de cinco macetas y un solo administrador, `docker logs` con `logrotate` sobre el daemon es suficiente. Si en el futuro se desplegaran múltiples invernaderos, se introduciría una capa de agregación.

7.6.2 Backups de PostgreSQL

No hay backups automatizados en v1.0 (deuda declarada del proyecto). El procedimiento manual recomendado es:

Listing 7.8: Backup manual semanal de la base de datos

```
# Backup
docker exec demeter-db pg_dump -U postgres demeter_db \
  | gzip > backup-$(date +%Y%m%d).sql.gz

# Restore (en una VM nueva o tras desastre)
gunzip -c backup-YYYYMMDD.sql.gz \
  | docker exec -i demeter-db psql -U postgres demeter_db
```

Para v1.0 se planifica una entrada `cron` semanal que escriba a un USB externo conectado a la Raspberry Pi (Semana 4 del plan de release).

7.6.3 Diagnóstico y troubleshooting

Los problemas operativos recurrentes del proyecto están documentados en archivos dedicados dentro de docs/:

Cuadro 7.5: Documentos de troubleshooting por capa

Documento	Cubre
Backend_Problemas_y_Soluciones.md	Errores comunes del API y la DB.
Firmware_Problemas_y_Soluciones.md	Errores de flasheo, ESP-NOW y handshake.
Raspberry_Problemas_y_Soluciones.md	UART, permisos serie, conexión WSS.
Error_Redis_Connection.md	Conexión de la API a Redis.
Error_Frontend_WS_UART.md	Errores WS y UART vistos desde el frontend.
Guia_Monitorizacion_TUI.md	Monitorización ad hoc de PostgreSQL/Redis desde CLI.

7.7 Actualización y Rollback

7.7.1 Actualización

Staging: `make staging-down && git pull && make staging` (arranca tirando de las imágenes :latest de ghcr.io ya construidas por CI).

Producción: `ssh` a la VM, `git pull origin main`, repetir el comando `docker compose ... up -d --build`. Se asume ventana de mantenimiento acordada; el sistema no es *zero-downtime*.

Raspberry Pi: `make rpi-pull` hace `pull` de la imagen ARM64 más reciente y reinicia el stack. Tiempo típico de corte: 5–15 segundos.

7.7.2 Rollback

Las imágenes en ghcr.io están etiquetadas con el hash de commit, el nombre de la rama, el tag semver y la etiqueta fija 0.4. Para volver a una versión anterior basta con apuntar el override al tag deseado:

Listing 7.9: Rollback a una versión anterior

```
# Identificar el tag al que volver (e.g. sha-abc1234 o un semver previo)
DEMETER_BACKEND_IMAGE=ghcr.io/danielmf31/proyecto_demeter/backend:sha-abc1234 \
DEMETER_FRONTEND_IMAGE=ghcr.io/danielmf31/proyecto_demeter/frontend:sha-abc1234 \
make staging
```

Consideración Importante

Migraciones no reversibles. Si entre la versión actual y la versión a la que se quiere volver hay una migración Alembic destructiva (e.g. una columna borrada), el rollback de la imagen no basta: hay que restaurar también la base de datos desde el backup más reciente. Por esa razón se recomienda hacer `pg_dump` antes de cualquier despliegue de producción que incluya migraciones.

7.8 Resumen

El despliegue v1.0 del sistema Demeter sostiene cuatro entornos (*dev*, *staging*, *producción*, *Raspberry Pi*) sobre una única familia de imágenes construidas por CI y publicadas en ghcr.io. La operación se gobierna desde un **Makefile con ~50 targets** que cubren desde el flasheo del firmware hasta la promoción manual a producción. Las decisiones de scope explícitas (sin OTA, sin auto-deploy, sin observabilidad centralizada, sin backups automáticos) corresponden a la realidad de un sistema doméstico con un administrador único; cada una está documentada como tal y pasa a deuda formal si en una futura versión se ampliara el alcance del despliegue.

Capítulo 8

Conclusiones y Cierre de la Versión 1.0

8.1 Estado de la Versión 1.0 Entregada

Demeter v1.0 se entrega como un sistema **completo en su arquitectura y verificado en banco**, modesto en alcance y nunca puesto en explotación real. Se archiva como *release* congelada: un artefacto estable, documentado y reproducible, con una cláusula explícita de no continuidad.

Cuadro 8.1: Inventario factual de lo entregado en la versión 1.0.

Artefacto	Estado al cierre
Capas implementadas	4 de 4: firmware ESP32-S3, servicio de borde en Raspberry Pi, backend en contenedores y SDK científico
Código propio	27.025 líneas, excluidas dependencias y ficheros generados
Protocolo	14 tipos de mensaje operativos sobre trama binaria propia, con handshake y validación por capas
Tests automatizados	145, repartidos en cinco suites (véase el Capítulo 6)
Documentación técnica	Este documento, 84 páginas, 18 diagramas y 9 cuadros de Decisión de Diseño
Imágenes Docker	Backend y frontend publicadas en GitHub Container Registry
Entornos de despliegue	5 configuraciones de Docker Compose, levantables desde un árbol limpio
Días de operación continua sin intervención	0

Esa última fila es la que da la medida honesta del cierre y merece leerse junto a todas las anteriores. El sistema **funciona**: la cadena completa, desde un sensor que despierta de *deep sleep* hasta el dato visible en el dashboard, se recorrió de extremo a extremo sobre tres nodos reales. Pero funcionó en un banco de laboratorio y bajo supervisión, nunca desatendido durante días ni expuesto a las condiciones de un invernadero. Demeter v1.0 es un **prototipo validado**, no un sistema en producción, y el resto de este capítulo se escribe desde esa premisa.

Cuadro 8.2: Alcance entregado frente a alcance excluido deliberadamente.

Entregado en v1.0	Fuera de alcance por decisión
Telemetría ambiental y de suelo por nodo y por planta	Cifrado del enlace de campo (AES-128 sobre ESP-NOW)
Actuación remota sobre GPIO y secuencias programadas	Actualización de firmware por aire (OTA)
Trazabilidad LIMS de plantas y experimentos	Multi-invernadero sobre una misma instancia
SDK con cálculo agronómico y exportación	Observabilidad centralizada y sistema de alarmas
Despliegue reproducible en cinco entornos	Interfaz adaptada a móvil

8.2 Cumplimiento de Objetivos

El Capítulo ?? declaró cuatro objetivos específicos. Su estado real al cierre es el siguiente.

Cuadro 8.3: Grado de cumplimiento de los objetivos declarados.

Objetivo	Estado	Evidencia y matices
Automatización IoT	Cumplido parcialmente	Tres nodos ESP32-S3 operativos en banco, con ciclo de <i>deep sleep</i> y reporte automático. La recolección «continua y sin intervención humana» que enunciaba el objetivo nunca se sostuvo más allá de una sesión de pruebas.
Control Remoto	Cumplido parcialmente	La cadena de mando completa opera para SET_GPIO, y el secuenciador ejecuta secuencias multi-paso. Pero no se actuó nunca sobre una bomba real : la instalación hidráulica se adquirió y no llegó a montarse, y el <i>spike</i> que debía validarla no se ejecutó. Este es el objetivo que queda más lejos de lo prometido.
Trazabilidad LIMS	Cumplido	Modelo de 13 tablas con relación N:M entre plantas y experimentos, consulta de telemetría por planta y registro de actividad (Capítulo 4).
Análisis Científico	Cumplido	SDK con caché Parquet, pipeline ETL, cálculo agronómico vectorizado y exportación a CSV y Excel (Capítulo 5), utilizable sin conocimientos de la infraestructura.

El patrón es nítido y conviene nombrarlo: **los dos objetivos de software puro se cumplieron; los dos que tocaban el mundo físico se quedaron a medias**. No es casualidad. El software se puede escribir de noche, en solitario y sin depender de nadie; montar una instalación de riego en un invernadero de otra escuela exige coordinación, permisos, presencia física y manos ajenas. La barrera no fue técnica.

8.3 Decisiones Arquitectónicas en Retrospectiva

Decisiones que se sostienen

- **Protocolo binario propio en lugar de JSON sobre el enlace de campo.** La trama compacta hizo viable el *deep sleep* agresivo y mantuvo la ocupación del canal por nodo por debajo del 0,01 %. El coste — tener que escribir y depurar el empaquetado — se amortizó con la validación por capas, que atrapa tramas corruptas antes de que lleguen a la lógica de dominio.
- **Pydantic como puente único entre lo binario y el dominio.** Definir el contrato una sola vez en `Software/Common` y consumirlo desde el backend y desde la Raspberry Pi eliminó de raíz toda una clase de errores de desincronización entre extremos. Es, con diferencia, la decisión que más problemas evitó.
- **Patrón *Strategy* sobre la interfaz `IComms`.** Permitió probar la lógica de protocolo en el PC, sin hardware, y explica que la suite de firmware sea la segunda más poblada del proyecto.
- **Imagen base común para nube y Raspberry Pi.** La misma definición de servicios sirvió para cinco entornos, con las diferencias confinadas a ficheros de *override*.
- **Caché Parquet local en el SDK.** Hace que el análisis iterativo no golpee la API en cada ejecución, que es exactamente el patrón de trabajo de quien explora datos.

Decisiones que se tomarían de otra forma

- **Validar el hardware crítico antes de comprarlo.** El *spike* de la bomba se escribió y nunca se ejecutó, y para cuando se adquirió la instalación hidráulica su plan de pruebas ya estaba obsoleto: contemplaba una bomba de 5 V y el material comprado era de 12 V. Un *spike* que no se ejecuta no reduce ningún riesgo; solo documenta que se era consciente de él.
- **Medir desde el primer día.** El sistema se dio por bueno sin línea base de latencia, pérdida de tramas ni autonomía real. Sin esas medidas no hay forma de demostrar que funciona ni de detectar una regresión, y añadirlas al final ya no es posible.
- **Integración continua completa desde el primer *release*.** El *pipeline* construye y publica las imágenes de backend y frontend, pero nunca compiló el firmware ni ejecutó análisis de secretos. La auditoría del cierre encontró una credencial viva versionada en el repositorio, que un análisis automático habría detectado el día que se introdujo.
- **Higiene del repositorio desde el inicio.** Versionar `node_modules`, la documentación generada por Doxygen y ficheros de *log* infla el repositorio en un orden de magnitud y arruina cualquier métrica automática sobre él. Corregirlo después obliga a reescribir el histórico.

Sobre las decisiones de arquitectura y su registro

El plan original contemplaba formalizar entre cinco y siete *Architecture Decision Records* como documentos independientes. **No se escribió ninguno.** En la práctica su función la cumplen los **nueve cuadros de Decisión de Diseño** repartidos por este documento, que recogen el mismo contenido — decisión, alternativas descartadas y justificación — pero incrustado en el capítulo donde la decisión es relevante en lugar de en un registro aparte.

La lección no es que los ADR sobren, sino que **un formato que nadie mantiene es peor que uno más humilde que sí se usa.** Para un proyecto de un solo autor, el cuadro dentro del capítulo resultó ser el vehículo que sobrevivió; el registro separado se habría quedado, como tantas otras cosas, en la lista de pendientes.

8.4 Lecciones Aprendidas

Técnicas

- **El contrato compartido vale más que cualquier optimización.** Un único modelo de datos consumido por todas las capas evitó la clase de fallo más cara de depurar en un sistema distribuido: dos extremos que creen hablar el mismo idioma y no lo hacen.
- **asyncio recompensa la disciplina y castiga la improvisación.** Cuatro tareas cooperativas sobre un solo hilo bastaron para el servicio de borde, pero cualquier llamada bloqueante que se cuele detiene todo el proceso. El modelo obliga a pensar dónde está cada punto de espera.
- **ESP-NOW resuelve el enlace de campo mejor de lo esperado, y con un límite claro.** No necesita punto de acceso, el consumo es compatible con el *deep sleep* y el alcance fue suficiente. Su límite es la seguridad: sin cifrado, cualquier receptor cercano lee la trama.
- **El binario merece la pena solo cuando la energía es el recurso escaso.** En el enlace de campo, sí. Entre la Raspberry Pi y el servidor, donde ya había JSON sobre WebSocket, habría sido una complicación gratuita.

De proceso

- **Documentación como código.** Mantener el LaTeX en el mismo repositorio, versionado junto al código que describe, es lo que ha hecho posible que este documento siga siendo cierto en el momento del cierre. Una documentación fuera del repositorio habría divergido en semanas.
- **Un *spike* sin ejecutar es una deuda disfrazada de diligencia.** Escribir la exploración y no hacerla da la sensación de haber gestionado el riesgo sin haberlo tocado. Es peor que no escribirla, porque tranquiliza.
- **Cerrar es una decisión de ingeniería, no una rendición.** La alternativa realista a este cierre no era un proyecto que siguiera creciendo, sino uno que siguiera facturando infraestructura sin que nadie lo tocara. Decidir el final y documentarlo produce un artefacto referenciable; dejarlo morir por inercia no produce nada.

Personales

El aprendizaje más incómodo del proyecto no es técnico. Demeter demostró que una sola persona puede construir un sistema distribuido de cuatro capas, documentarlo en ochenta páginas y desplegarlo de forma reproducible. Demostró también que **esa misma persona no puede sostenerlo**, y que la parte que exigía coordinarse con otros — montar la instalación en el invernadero, medir en campo, transferir el conocimiento — fue precisamente la que no se hizo.

La conclusión que el autor se lleva al siguiente proyecto es que la capacidad técnica individual no es el cuello de botella, y que optimizarla más no habría cambiado el resultado. Lo que faltó fue todo lo demás.

8.5 Trabajo Futuro y Extensiones Posibles

Las siguientes extensiones son plausibles sobre la arquitectura entregada. Se inventarían aquí para quien retome el trabajo; **ninguna constituye un compromiso** de la versión 1.0 ni del autor.

- **Caracterización en campo.** Latencia extremo a extremo, pérdida de tramas, autonomía real y tiempo de reconexión. Es el primer trabajo que debería hacer cualquier continuación: sin línea base no se puede saber si un cambio mejora o empeora el sistema.

- **Montaje y validación de la instalación de riego.** El material está adquirido y sin usar. Requiere reescribir el *spike* para 12 V y dimensionar la conmutación: la fuente disponible no admite más de una bomba simultánea.
- **Cifrado del enlace de campo.** AES-128 sobre el *payload* ESP-NOW, imprescindible antes de cualquier despliegue con datos que importen.
- **Observabilidad centralizada.** Agregación de *logs* y métricas. Hoy diagnosticar una incidencia obliga a leer la salida de cada contenedor por separado.
- **Alta de nodos sin recompilar.** La dirección del *gateway* y el identificador de nodo se fijan en tiempo de compilación; añadir un nodo exige reflashear.
- **Actualización de firmware por aire.** Con autenticación y *rollback*. Se descartó en v1.0 por alcance, pero condiciona cualquier despliegue de más de un puñado de nodos.
- **Riego predictivo.** Integración con datos meteorológicos o una capa de inferencia por especie. Es la extensión más vistosa y la que menos sentido tiene abordar antes que las cinco anteriores.

8.6 Cierre y Agradecimientos

Con la publicación de este documento, el Proyecto Demeter queda **archivado en su versión 1.0**. No hay desarrollo planificado. El repositorio se abre con licencia MIT para el código y CC-BY-SA 4.0 para la documentación, de modo que cualquiera pueda estudiarlo, reutilizarlo o continuarlo por su cuenta, sin que ello comprometa al autor ni a la asociación a mantenimiento alguno.

El análisis que sostiene esta decisión — económico, de recursos humanos y técnico — se recoge en el documento hermano *Informe de Viabilidad y Propuesta de Cierre*, en [docs/Closure/](#).

Quedan los agradecimientos. A la **Asociación Universitaria ESIBot**, por el contexto, los recursos y la libertad para llevar el proyecto hasta donde llegó. Al **Departamento de Agronomía de la ETSIA**, por plantear un problema real y confiar en que un grupo de estudiantes de otra escuela pudiera abordarlo. Y a quienes, en algún momento de estos veinte meses, preguntaron cómo iba: sostener un proyecto en solitario se parece bastante a hablar solo, y esas preguntas contaban más de lo que probablemente parecía.

Decisión de Diseño

Por qué se archiva como *release* y no como abandono. La diferencia es material. Un proyecto abandonado deja de ser operable: nadie sabe si arranca, la documentación miente y el conocimiento se disipa. Una versión archivada es un artefacto estable, reproducible desde un árbol limpio, documentado hasta sus limitaciones y publicado con licencia clara. Demeter cumple los criterios de la segunda categoría, incluido el más incómodo: dejar escrito, con nombres y cifras, todo aquello que no llegó a funcionar. Un documento que solo contara los aciertos sería propaganda; y la propaganda, a diferencia de un artefacto, no le sirve a nadie que venga después.

Apéndice A

Diagramas del Protocolo Binario Dementer

Esta sección recoge los diagramas de flujo completos del protocolo de comunicación binario implementado entre los nodos ESP32 y el Gateway.

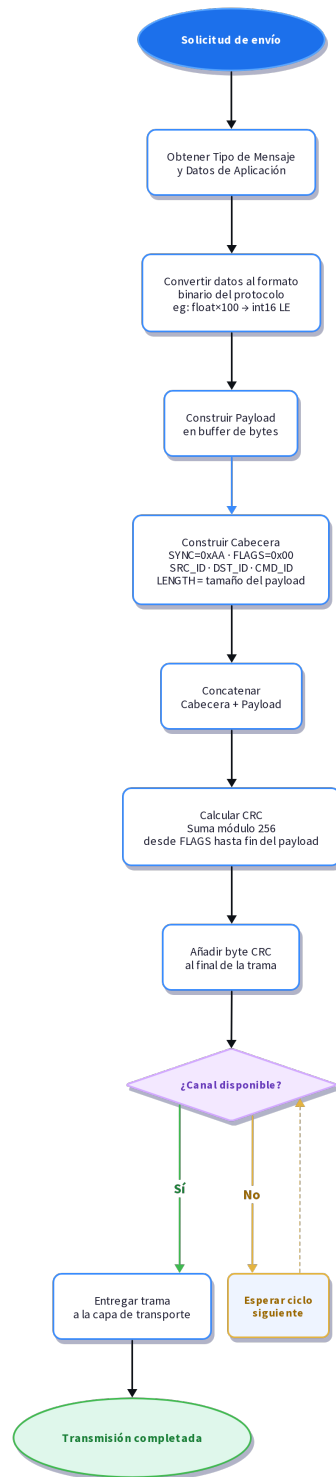


Figura A.1: Flujo de serialización de una trama binaria Demeter.

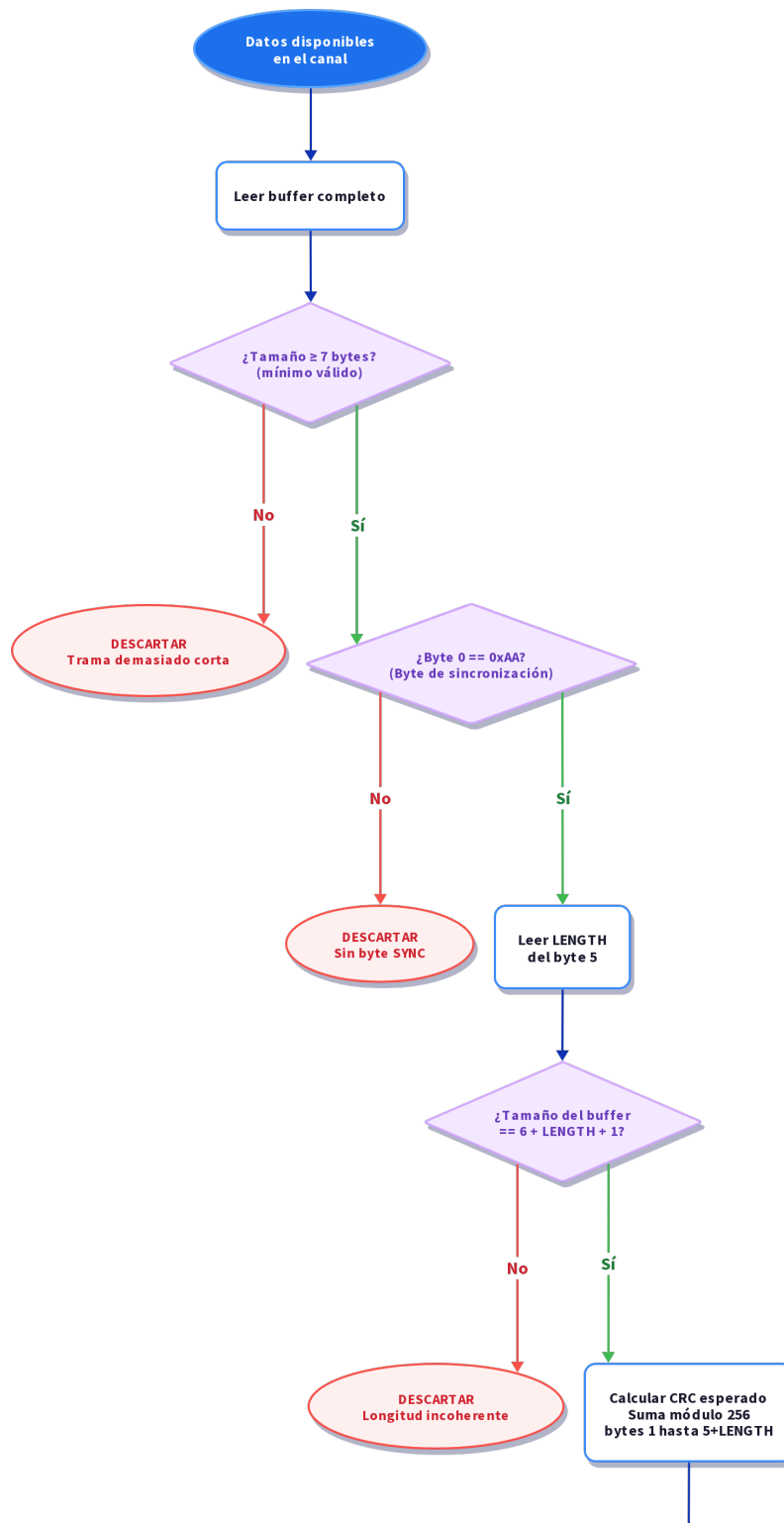


Figura A.2: Flujo de deserialización y validación de una trama recibida (parte superior).

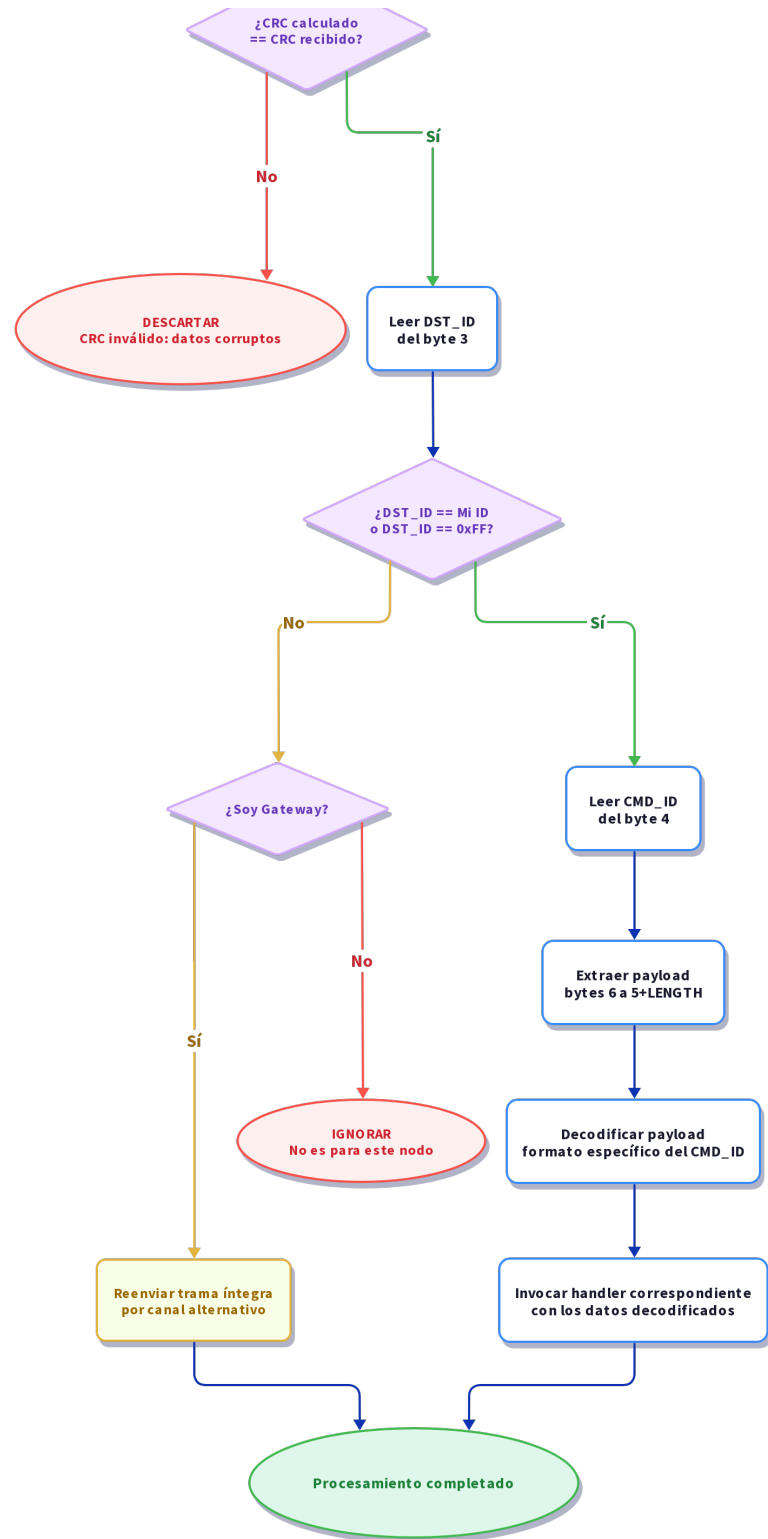


Figura A.3: Flujo de deserialización y validación de una trama recibida (parte inferior).

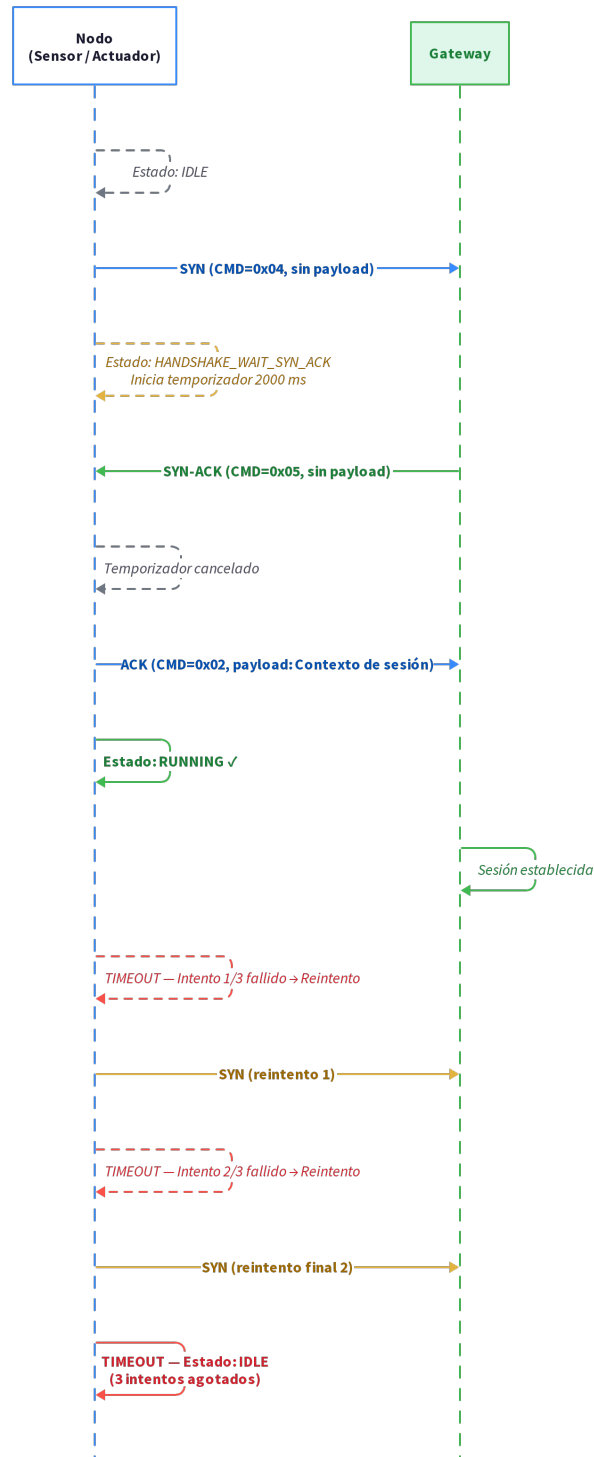


Figura A.4: Diagrama de secuencia del handshake SYN / SYN-ACK / ACK con reintentos.

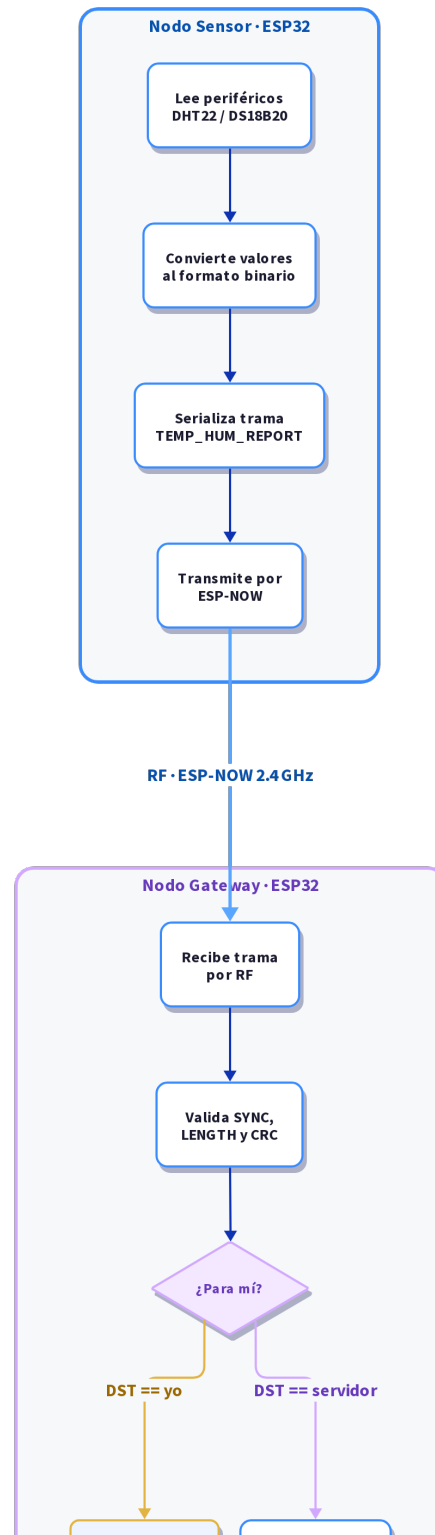


Figura A.5: Flujo end-to-end: Nodo Sensor → Gateway → Raspberry Pi → Backend (parte superior).

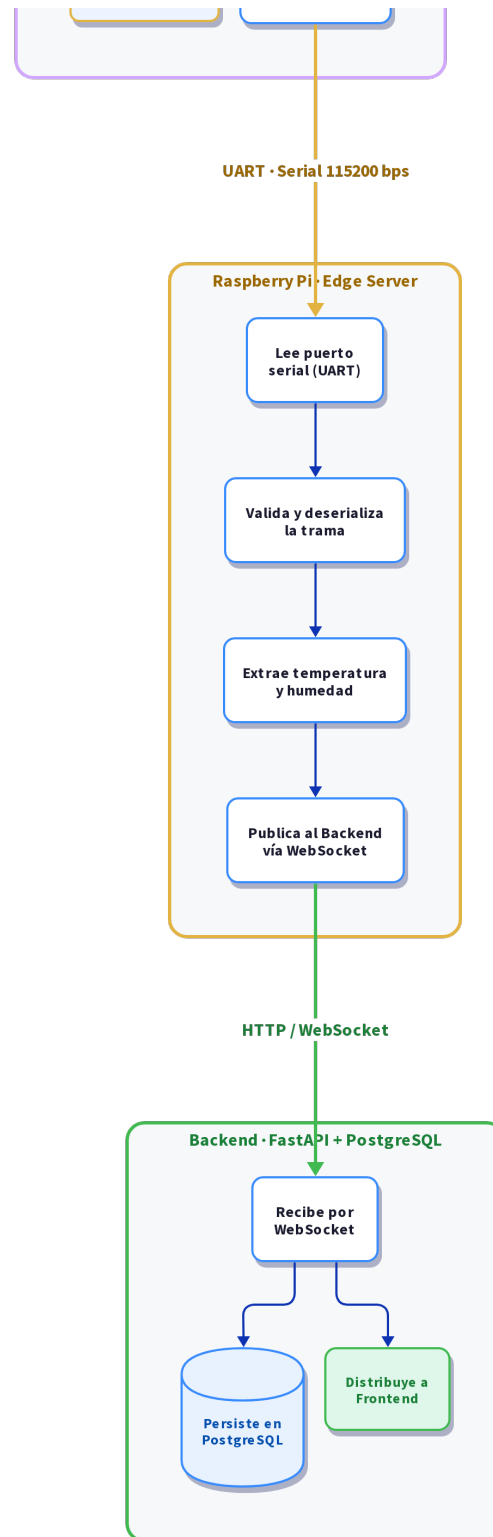


Figura A.6: Flujo end-to-end: Nodo Sensor → Gateway → Raspberry Pi → Backend (parte inferior).

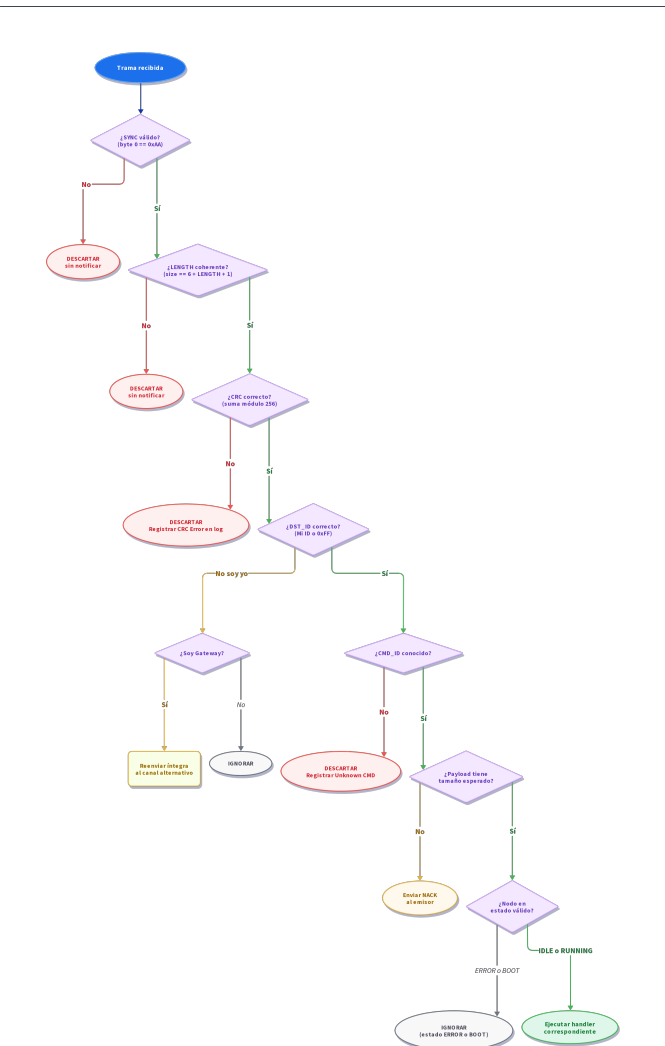


Figura A.7: Árbol de decisión para el manejo de errores en la recepción de tramas.

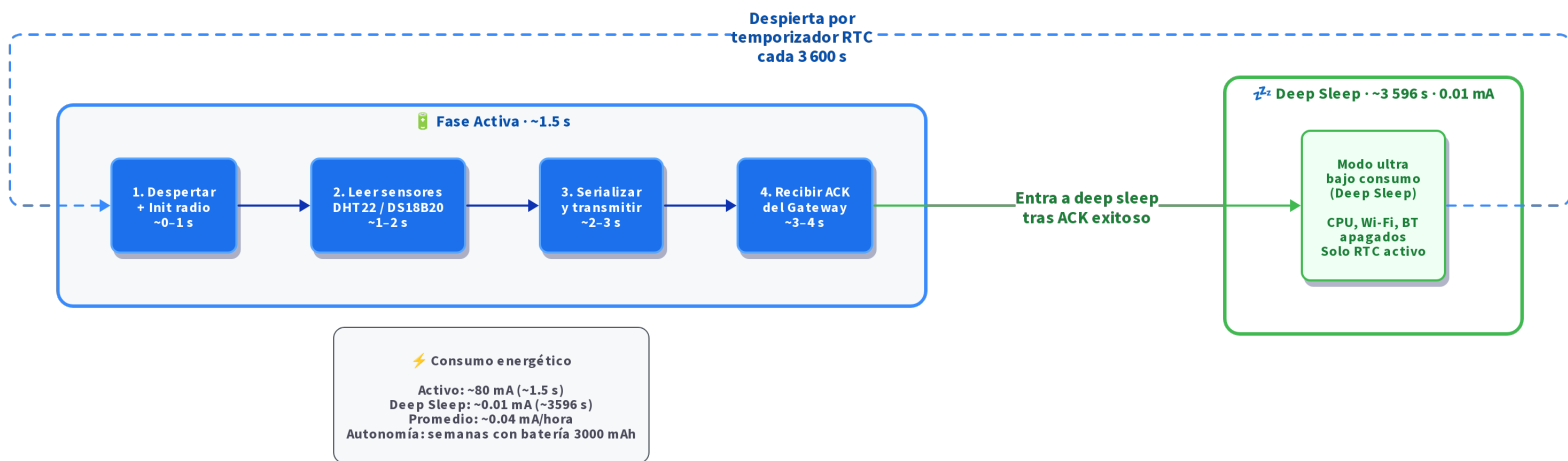


Figura A.8: Ciclo energético del nodo sensor: fase activa (~1,5 s) y Deep Sleep (~3596 s).

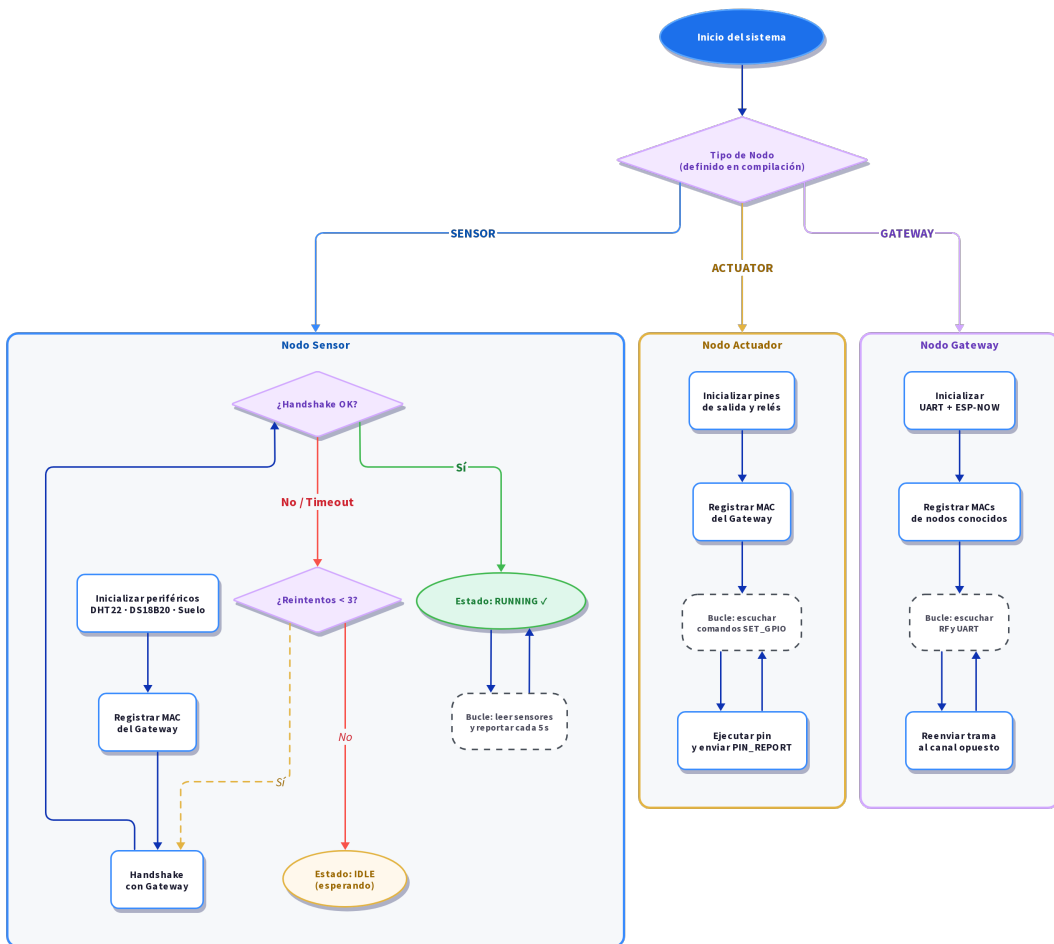


Figura A.9: Ciclo de vida de los nodos ESP32 según su rol: Sensor, Actuador o Gateway.

Apéndice B

Arquitectura del Servicio Raspberry Pi

Este anexo detalla la arquitectura del servicio Python ejecutado en la Raspberry Pi, que actúa como pasarela de borde (*edge gateway*) entre la red de nodos ESP32 y el servidor central. El servicio se ejecuta en un contenedor Docker con acceso directo al puerto serie GPIO (`/dev/serial0`) y gestiona cuatro tareas AsyncIO concurrentes: transporte UART, cliente WebSocket, servidor HTTP local (FastAPI en el puerto 8001) y el despachador de comandos. Los diagramas de este anexo cubren la arquitectura de componentes, los flujos de telemetría y comandos, el despliegue Docker y las tareas asíncronas.

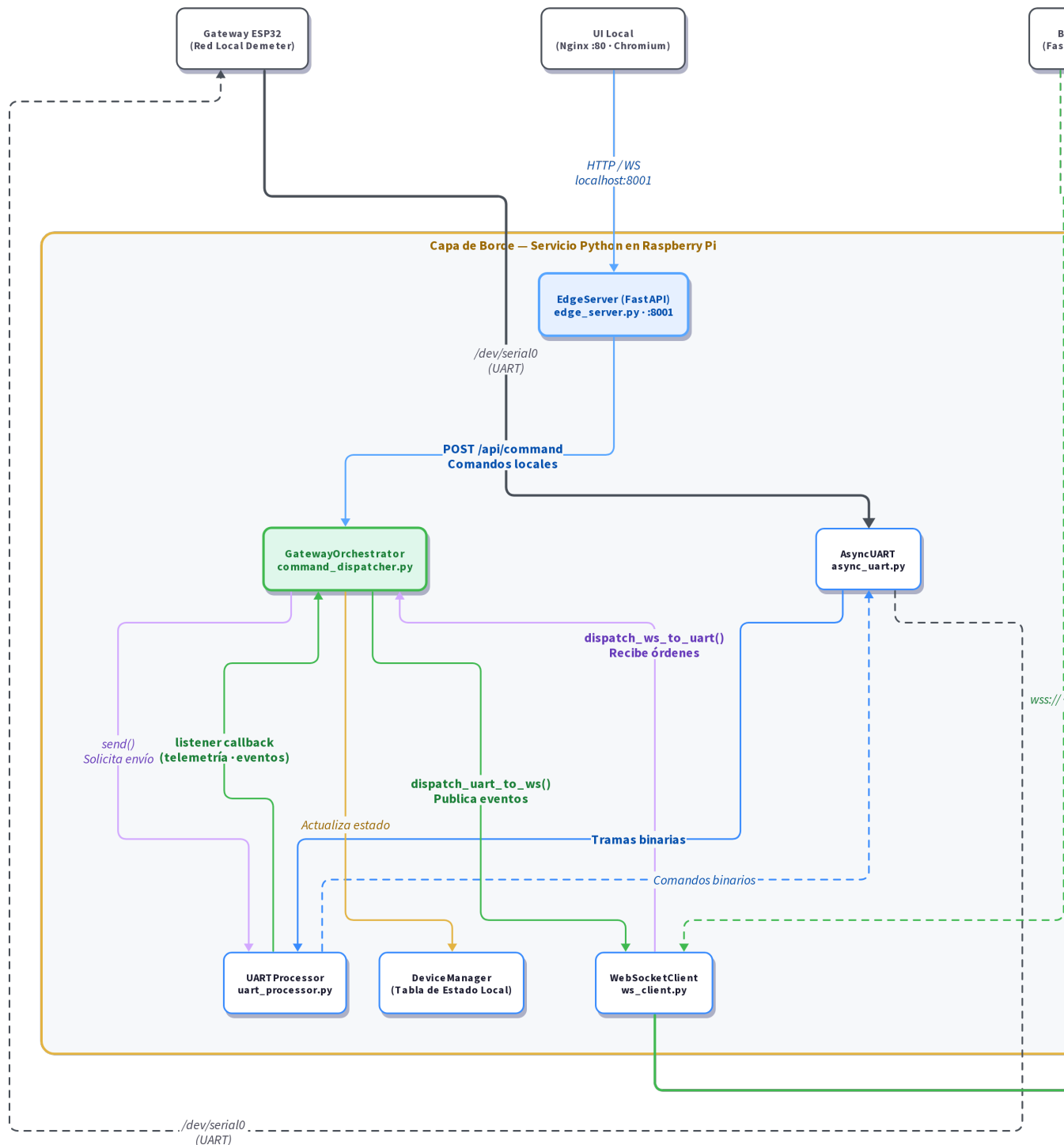


Figura B.1: Arquitectura de componentes del servicio Python en la Raspberry Pi (capa de borde).

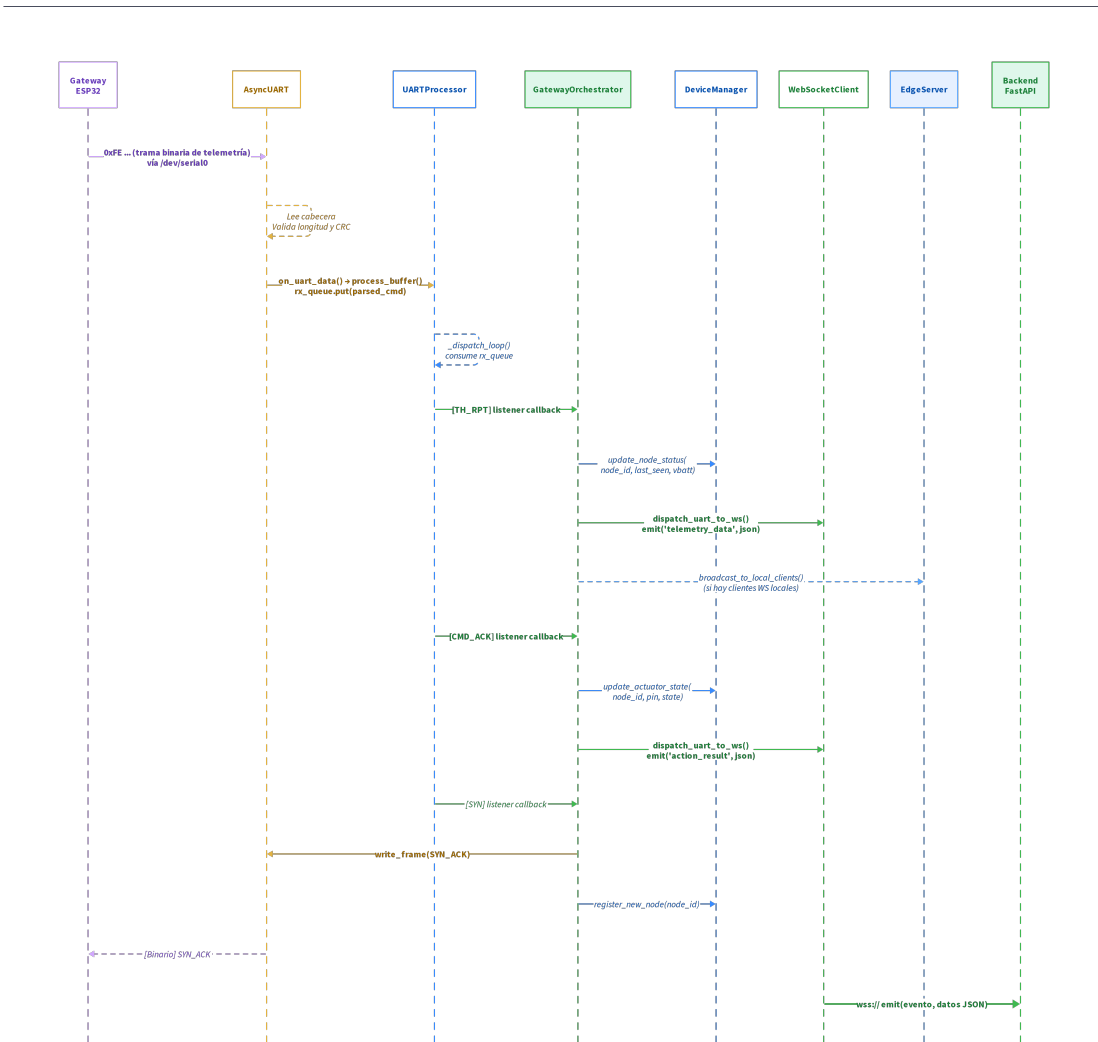


Figura B.2: Diagrama de secuencia: flujo ascendente de telemetría desde el Gateway hasta el Backend.

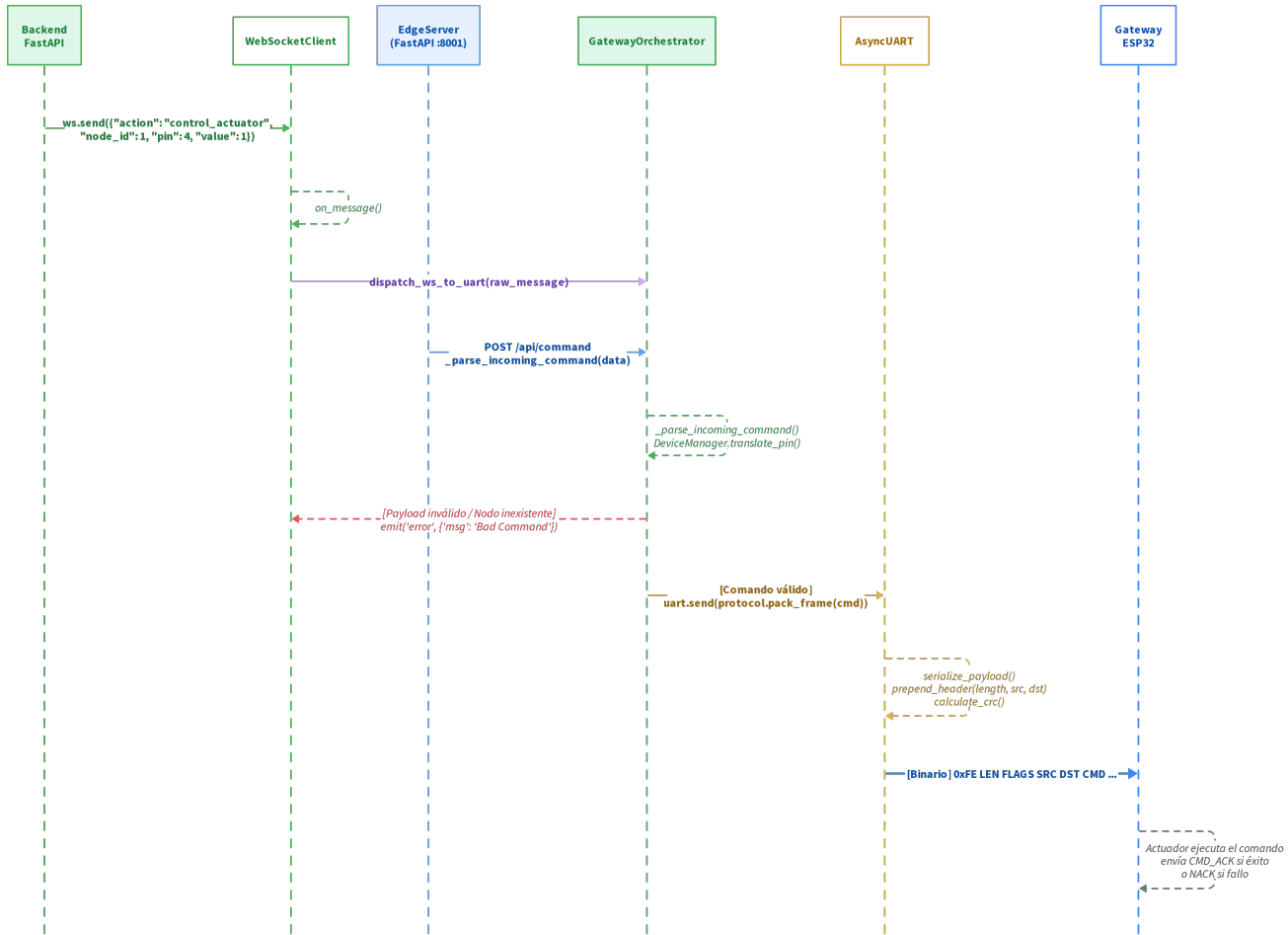


Figura B.3: Diagrama de secuencia: flujo descendente de comandos desde el Backend hasta el Gateway ESP32.

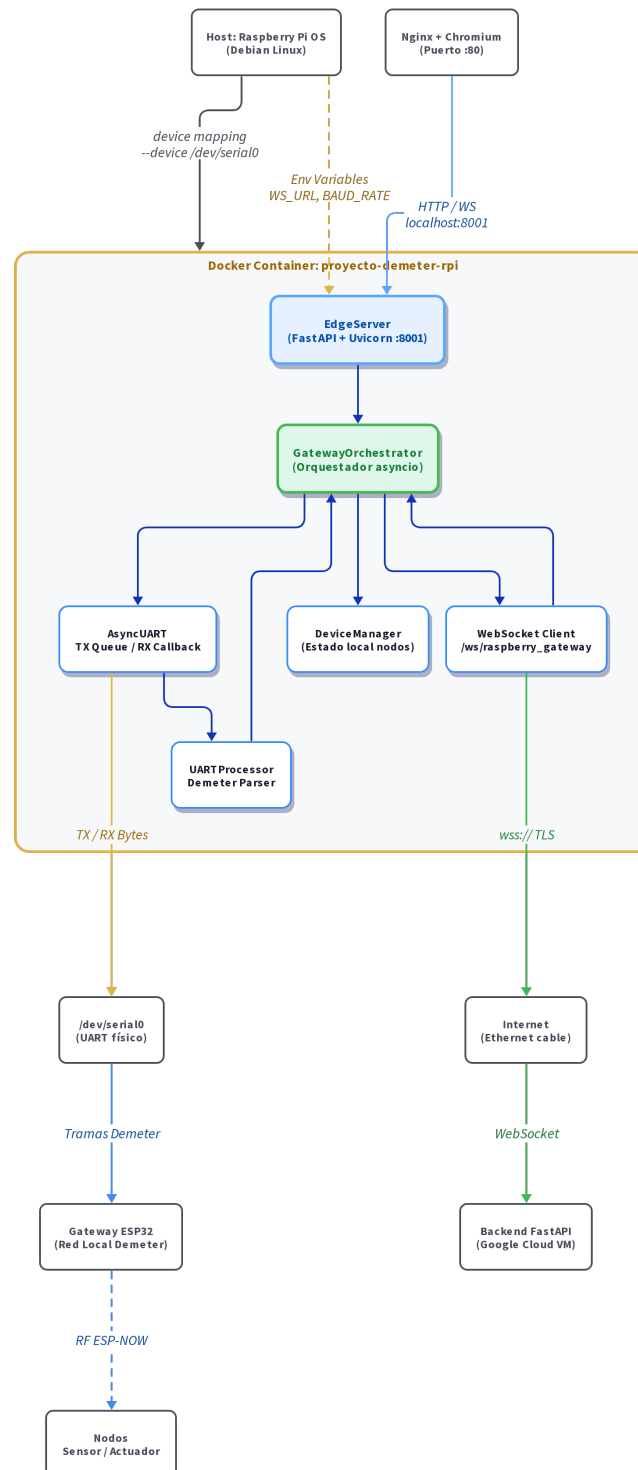


Figura B.4: Despliegue del servicio Raspberry Pi en contenedor Docker con sus conexiones externas.

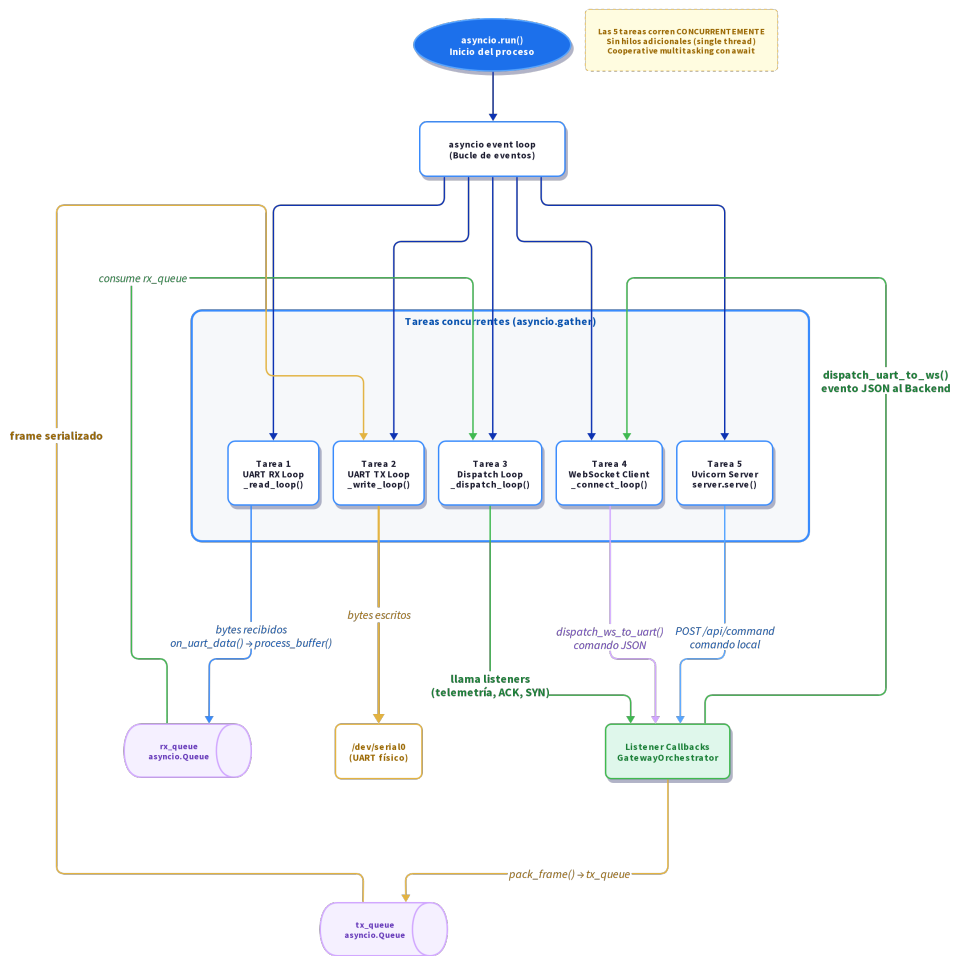


Figura B.5: Diagrama de las cuatro tareas AsyncIO concurrentes del servicio Raspberry Pi.

Apéndice C

Arquitectura del Backend

Este anexo presenta los diagramas de la arquitectura del servidor central desplegado en Google Cloud Platform. El backend se compone de seis contenedores Docker (API FastAPI, frontend Nginx, PostgreSQL, Redis, math-worker RQ y secuenciador) interconectados a través de la red privada **demeter-net**. La comunicación con la Raspberry Pi se realiza mediante WebSocket y el despacho de comandos utiliza el patrón Redis Pub/Sub descrito en el Capítulo [4](#).

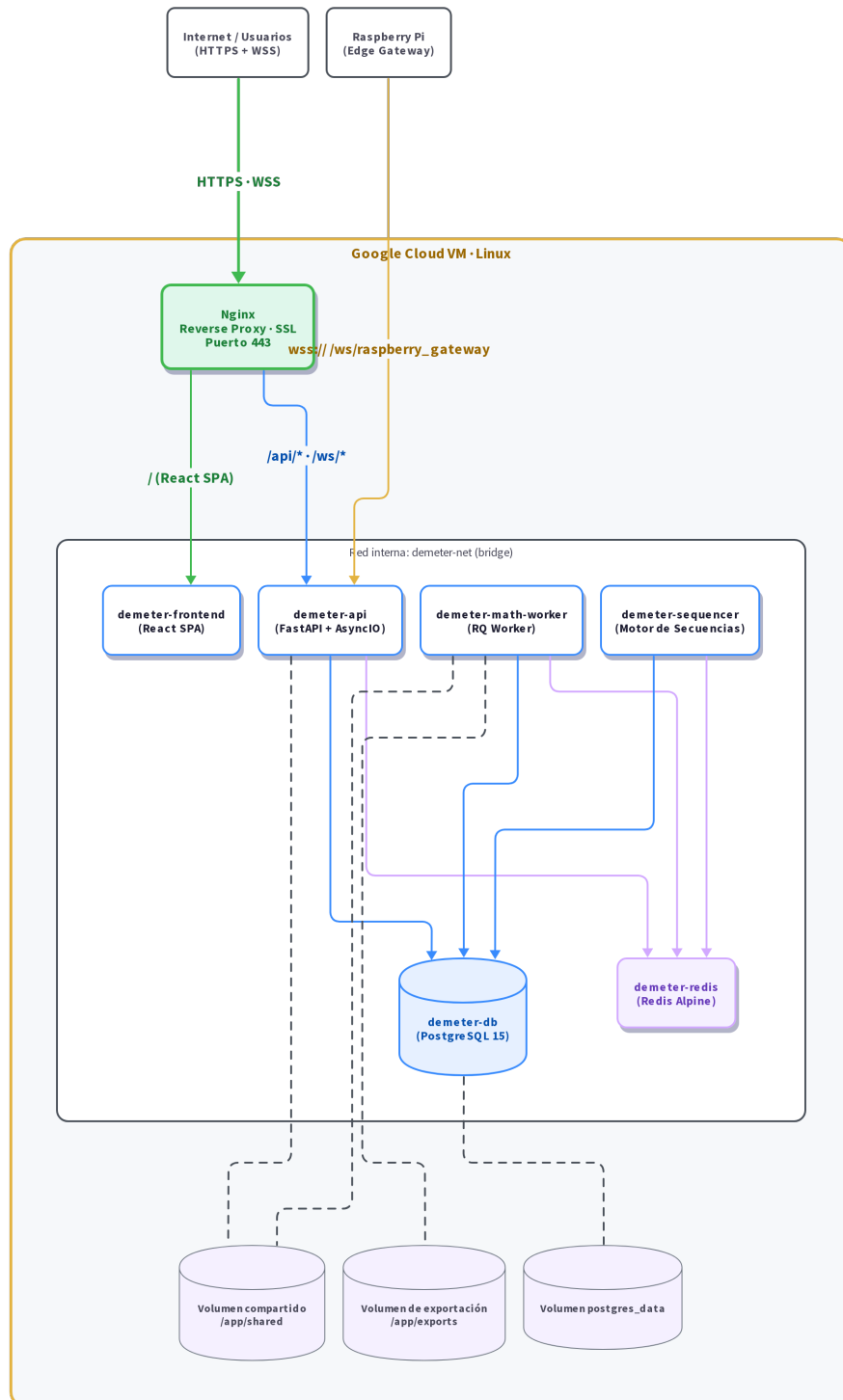


Figura C.1: Arquitectura de contenedores Docker del Backend en Google Cloud VM.

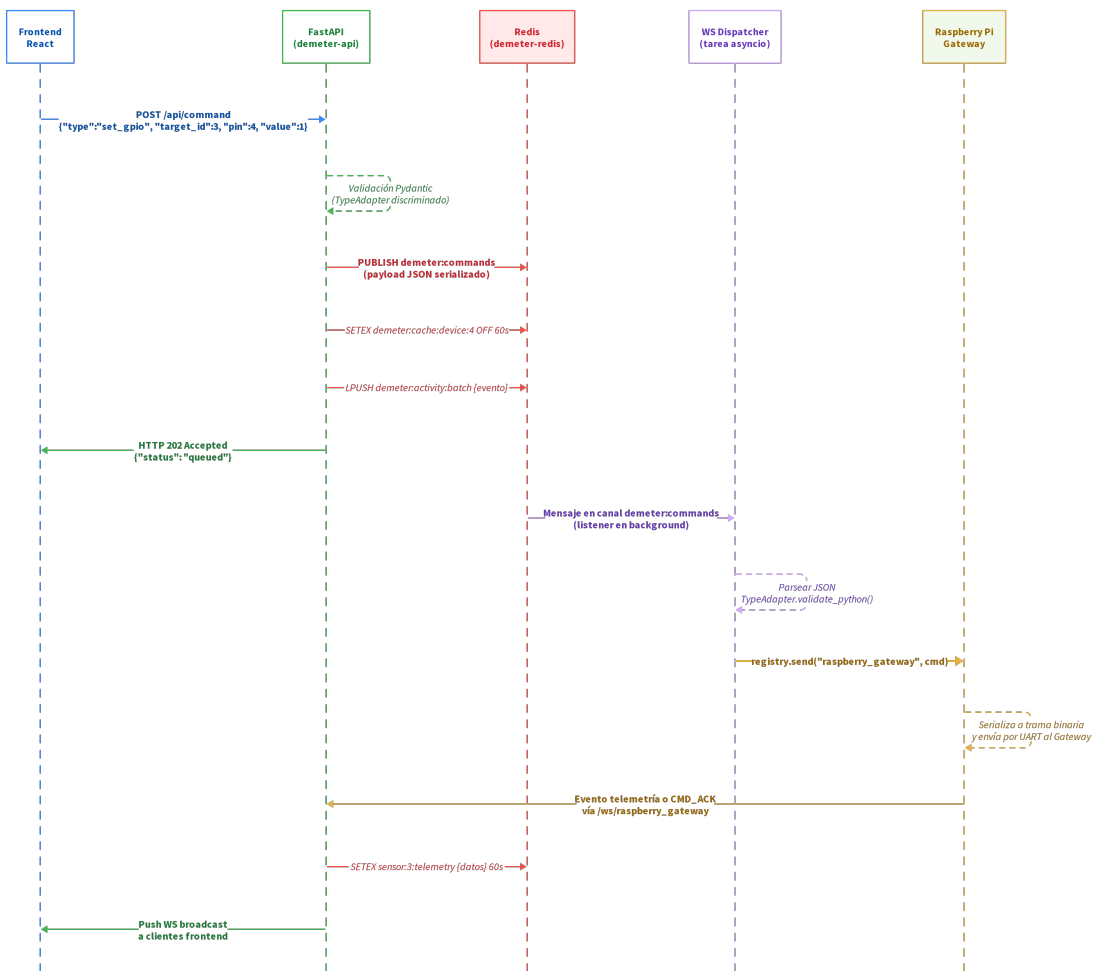


Figura C.2: Flujo de un comando de usuario a través del sistema Redis Pub/Sub hasta la Raspberry Pi.

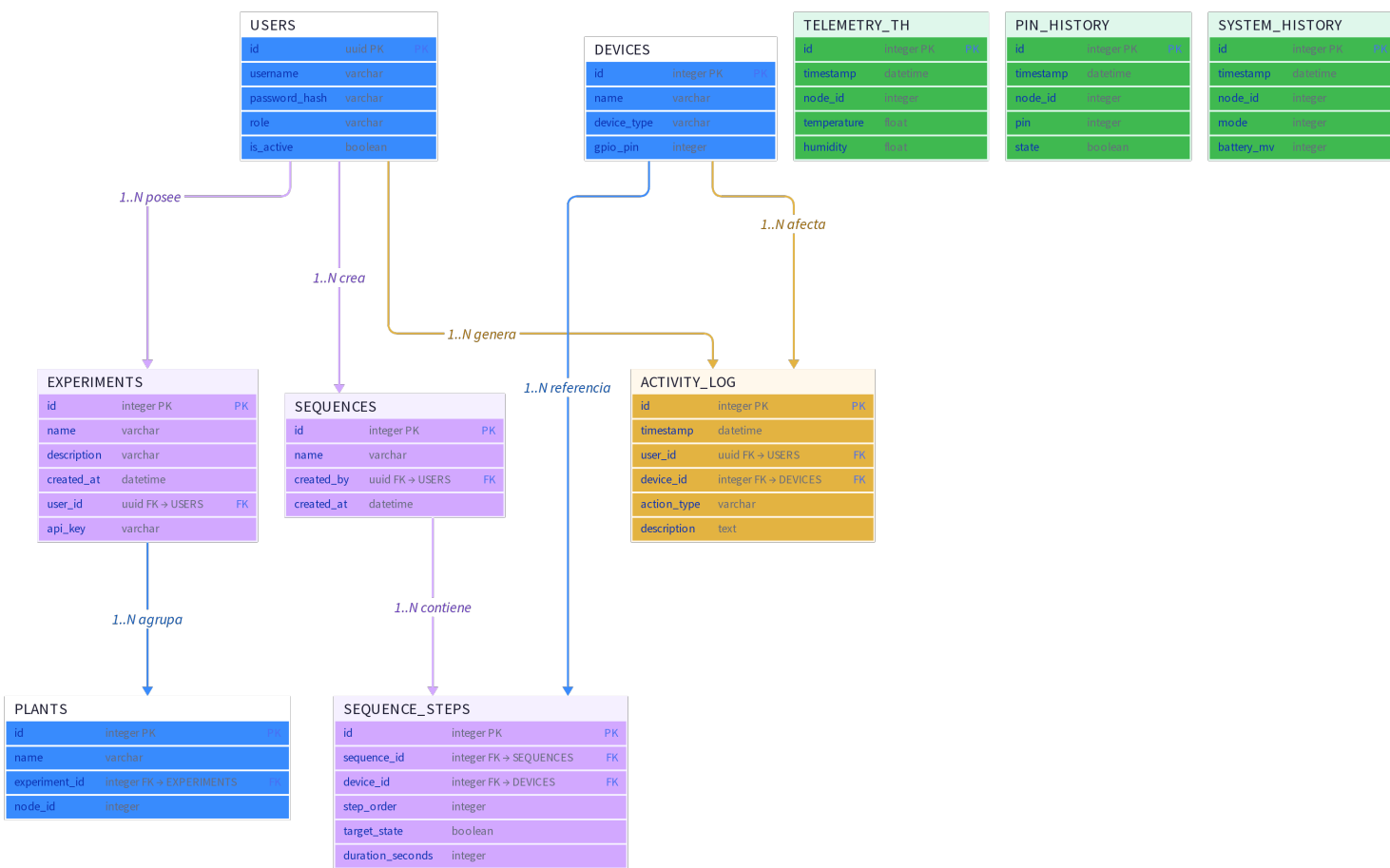


Figura C.3: Esquema entidad-relación de la base de datos PostgreSQL del Backend.